

COMP4801 Final Year Project Report

Exploring Optimization Opportunity for WhatsHap

Ng Ching Lap

UID: 3035687742

Supervisor: Professor Luo, Ruibang

Contents

Abstract	viii
1. Introduction	1
1.1 Project goal	1
1.2 Contributions	1
1.3 Report organization	1
2. Background	3
2.1 Long-read sequencing and ONT-like characteristics	3
2.2 WhatsHap in practical pipelines (VCF + BAM → phased VCF)	3
2.3 Why end-to-end benchmarking is necessary	3
2.4 Metrics	4
3. Functional Requirements (FRS-style)	5
3.1 Product perspective and scope	5
3.2 Users and operating environment	5
3.3 Naming conventions and artifacts (prefix-based)	5
3.4 Functional requirements	6
FR-1 Reference generation	6
FR-2 Ground-truth variant and haplotype generation	6
FR-3 ONT-like read simulation	7
FR-4 Read alignment	7
FR-5 Variant calling	8
FR-6 WhatsHap phasing (vendored integration)	8
FR-7 Phasing evaluation against truth	8
FR-8 End-to-end reporting and aggregation	9
FR-9 Experiment driver for systematic studies	9
3.5 Non-functional requirements (supporting the FYP goals)	10

4. System Design (SDD-style)	11
4.1 Design objectives	11
4.2 High-level pipeline structure and tool mapping	11
4.2.1 Pipeline stages (conceptual view)	11
4.2.2 Implementation mapping (codebase view)	12
4.2.3 Stage boundaries and responsibilities (quality view)	13
4.3 Component decomposition and responsibilities	13
4.3.1 Reference generator (<code>dataset.longread.reference</code>)	13
4.3.2 Truth + haplotype generator (<code>dataset.longread.truth</code>)	14
4.3.3 Long-read simulator (<code>dataset.longread.readsim</code>)	14
4.3.4 Orchestration runner (<code>benchmark.longread_pipeline_runner</code>)	15
4.3.5 Unified phasing CLI (<code>algorithms.cli.phase</code>)	16
4.3.6 WhatsHap BAM driver (<code>algorithms.diploid.whatschap_bam_driver</code>)	16
4.3.7 Evaluation (<code>benchmark.vcf_phase_eval</code>)	17
4.3.8 Aggregation (<code>benchmark.aggregate_pipeline_reports</code>)	17
4.3.9 Experiment driver (<code>benchmark.experiment_driver</code>)	17
4.4 Data contracts and file-level interfaces	18
4.5 Realism knobs as configurable stressors	18
4.5.1 Stressor catalogue and where it applies	18
4.5.2 Measurement expectations (what changes when knobs increase)	19
4.6 Key design decisions (rationale)	20
4.7 Extensibility	20
5. Implementation	22
5.1 Repository structure (implementation map)	22
5.2 End-to-end orchestration: <code>benchmark.longread_pipeline_runner</code>	23
5.2.1 Pipeline execution steps	23
5.2.2 Toolchain validation and robustness	24
5.3 WhatsHap integration: <code>diploid-whats-bam</code>	24
5.3.1 Vended WhatsHap core	25

5.3.2 Read filtering and selection	25
5.3.3 Phased VCF output	25
5.4 Evaluation implementation: <code>benchmark.vcf_phase_eval</code>	25
5.5 Indel-aware policy: SNP-only phasing/evaluation flags	26
5.6 Experiment automation and aggregation	26
5.6.1 Experiment driver	26
5.6.2 Aggregation contract	26
5.7 Realism knobs: where they are implemented and how they are surfaced	26
6. Testing and Validation (STD-style)	28
6.1 Testing objectives	28
6.2 Test levels and scope	28
6.3 Test environment and dependencies	29
6.3.1 Python environment	29
6.3.2 Vended WhatsHap core	29
6.3.3 External toolchain (system tests)	29
6.4 Implemented unit tests (current repository)	29
6.5 Implemented integration tests (current repository)	30
6.6 System / end-to-end smoke validation (long-read pipeline)	30
6.7 Validation for realism knobs (correctness criteria)	31
6.8 Coverage limitations and recommended additional tests	31
7. Software Configuration Management Plan (SCMP-style)	33
7.1 Configuration items (CIs)	33
7.2 Version control strategy	33
7.3 Change control and traceability	33
7.4 Artifact naming and storage policy	34
7.5 Baseline and release management	34
7.6 Backup and recovery	34

8. Software Quality Assurance Plan (SQAP-style)	35
8.1 Quality goals	35
8.2 Standards and conventions	35
8.3 Verification activities	35
8.4 Defect handling and regression prevention	36
8.5 Documentation QA	36
8.6 Limitations	36
9. Project Management Plan (PMP-style)	37
9.1 Deliverables	37
9.2 Work plan and milestones (final-phase focus)	37
9.3 Risks and mitigations	38
9.4 Tracking and reporting	38
10. Experimental Evaluation	39
10.1 Experimental methodology and common setup	39
10.1.1 Environment and toolchain	39
10.1.2 Pipeline regimes	39
10.1.3 Fixed parameters (baseline constants)	39
10.1.4 Metrics reported (from <code>aggregate.csv</code>)	41
10.2 Baseline: depth scaling (coverage proxy via <code>num_reads</code>)	42
Aim	42
Setup	42
Figures to include	42
Results summary (means over seeds)	44
Key observations	44
Takeaway	44
10.3 Single-knob study: duplicated regions (reference stressor)	45
Aim	45
Setup	45
Figures to include	45

Results summary (means over seeds)	46
Key observations	47
Takeaway	47
10.4 Single-knob study: coverage dropout (read start model)	48
Aim	48
Setup	48
Figures to include	48
Results summary (means over seeds)	49
Key observations	50
Takeaway	50
10.5 Single-knob study: correlated error bursts	51
Aim	51
Setup	51
Figures to include	51
Results summary (means over seeds)	52
Key observations	53
Takeaway	53
10.6 Single-knob study: read length model	54
Aim	54
Setup	54
Figures to include	54
Results summary (means over seeds)	55
Key observations	55
Takeaway	56
10.7 Truth indels and SNP-only policy (evaluation robustness)	56
Aim	56
Setup	56
Figures to include	57
Results summary (means over seeds)	58
Key observations	58

Takeaway	59
10.8 Interaction study: duplications $\times$ dropout	59
Aim	59
Setup	59
Figures to include	59
Results summary (means over seeds)	60
Key observations	61
Optimization implication	62
Takeaway	62
10.9 Optimization sweeps (WhatsHap / pipeline parameters)	62
Aim	62
Hard scenario	63
10.9.1 Sweep A: max_coverage	63
10.9.2 Sweep B: phasing quality thresholds (min_mapq $\times$ min_baseq)	64
10.9.3 Sweep C: calling thresholds (call_min_mapq $\times$ call_min_baseq)	66
10.9.4 Sweep D: runtime-focused lower max_coverage	68
10.9.5 Sweep E: fine phasing min_baseq	71
10.9.6 Default vs optimized comparison	73
10.9.7 Sweep F: local joint search around recommended thresholds	75
10.9.8 Sweep G: accuracy/runtime frontier comparison	77
10.9.9 Optimization robustness across scenarios	81
10.9.10 Sweep H: caller call_min_mapq rule-out	83
10.9.11 DP-scaling under hard conditions	85
Overall takeaway	87
10.10 Summary of results (cross-experiment synthesis)	88
10.10.1 Attribution summary (oracle vs called)	88
10.10.2 Most impactful stressors	89
10.10.3 Optimization summary	90

11. Discussion	92
11.1 Discussion of main findings	92
11.2 Practical implications for WhatsHap-based phasing	93
11.3 Limitations of the present study	94
11.4 Future work	95
11.5 Conclusion	95
12. Conclusion	96

Abstract

Haplotype phasing assigns heterozygous variants to their chromosome of origin and is a key step in genomic analysis. Long-read sequencing technologies (e.g., Oxford Nanopore Technologies, ONT) provide reads spanning many variants and can improve phasing contiguity. However, practical phasing performance depends on upstream alignment and variant calling, and on long-read characteristics such as non-uniform coverage, repetitive regions, and structured error patterns. WhatsHap is a widely used phasing tool for long reads; systematically understanding its end-to-end behavior under realistic conditions requires a controlled and reproducible benchmarking environment.

This project builds a reproducible benchmarking pipeline that generates a synthetic diploid reference genome, phased ground-truth variants and haplotypes, simulates ONT-like reads with configurable realism knobs, aligns reads, calls variants, phases using a vendored WhatsHap core, and evaluates outputs against truth. The pipeline supports both oracle and called variant sets to separate phaser-limited and caller-limited regimes. The implementation is validated with unit tests and an end-to-end test run, and the framework is then used for ablations and parameter sweeps to isolate failure modes and identify optimization directions for practical WhatsHap usage and long-read phasing workflows.

1. Introduction

Haplotype phasing assigns heterozygous variants to their chromosome of origin, enabling downstream analyses such as compound heterozygosity detection, allele-specific expression, and improved variant interpretation. Long-read sequencing provides reads that span multiple variants and can therefore improve both the contiguity and correctness of phased haplotypes compared to short-read data. In practice, however, phasing quality is not determined by the phasing algorithm alone: it is affected by read alignment quality, variant calling accuracy, and sequencing characteristics such as coverage bias, repetitive/duplicated regions, and structured error patterns.

WhatsHap is one of the most widely adopted phasing tools for long reads. While its algorithmic foundations are well studied, its behavior in practical end-to-end pipelines can vary significantly depending on upstream errors and data realism. A controlled benchmarking environment—where individual factors can be varied independently and runs can be reproduced reliably—is necessary to understand WhatsHap’s capabilities and limitations in realistic settings, and to identify opportunities for optimization.

1.1 Project goal

This project aims to:

1. Build a reproducible end-to-end benchmarking platform that mimics key characteristics of ONT-like long-read phasing workflows.
2. Evaluate WhatsHap’s phasing capability under increasingly realistic conditions and quantify how different factors degrade performance.
3. Identify optimization directions or configuration recommendations for practical WhatsHap usage.

1.2 Contributions

- An end-to-end long-read simulation and benchmarking pipeline producing reference, truth variants/haplotypes, reads, alignments, variants, phased outputs, and standardized JSON/CSV reports.
- A dual evaluation approach using **oracle** (ground-truth) and **called** variant sets to separate phasing limitations from variant-calling limitations.
- Implemented realism “knobs” (e.g., duplicated regions, dropout, correlated error bursts, indel-rich truth) to isolate impacts on alignment/calling/phasing.
- Automated experiment running, aggregation, and plotting to support systematic studies and report-ready results.

1.3 Report organization

Section 2 introduces background on long-read phasing, WhatsHap, and evaluation metrics. Sections 3–9 describe requirements, design, implementation, validation, configuration management, quality assurance, and project man-

agement. Section 10 presents experimental results and optimization investigations, followed by discussion and conclusions.

2. Background

2.1 Long-read sequencing and ONT-like characteristics

Long-read sequencing reads can span kilobases to megabases, often covering many heterozygous variants within a single read. This enables long-range phasing and reduces ambiguity compared to short reads. ONT data, however, exhibits properties that complicate downstream analysis, including higher raw error rates than short reads, a mixture of substitution and indel errors, potential error clustering (bursts), and non-uniform coverage due to sequencing and mapping biases. Repetitive and duplicated regions further introduce ambiguity in read mapping, which can degrade both variant calling and phasing.

2.2 WhatsHap in practical pipelines (VCF + BAM → phased VCF)

In a typical long-read workflow, reads are aligned to a reference to produce BAM/CRAM, variants are called to produce a VCF containing genotypes, and a phasing tool assigns heterozygous variants to haplotypes. WhatsHap takes aligned reads (BAM) and variants (VCF) and extracts read-backed allele observations at variant positions. It then partitions heterozygous variants into phase blocks and computes a phasing solution within each block under an error-correction objective. The output is a phased VCF where heterozygous genotypes are phased (0|1 / 1|0) and annotated with phase set identifiers (PS) indicating blocks.

2.3 Why end-to-end benchmarking is necessary

Phasing performance depends on multiple stages:

- **Alignment:** mapping quality, ambiguous mappings in repeats/duplications, and alignment artifacts around indels.
- **Variant calling:** missing true variants (recall loss), spurious variants (precision loss), and genotype errors.
- **Phasing:** block fragmentation and phase inconsistencies (switch errors) among phased variants.

To understand where failures originate, this project evaluates WhatsHap under two regimes:

- **Oracle VCF:** uses ground-truth variants to isolate phasing behavior given a correct variant set.
- **Called VCF:** uses variants produced by a caller to measure realistic end-to-end performance, including caller limitations.

This separation makes it possible to distinguish “phaser-limited” regimes (phasing errors dominate) from “caller-limited” regimes (missing/incorrect variants dominate).

2.4 Metrics

This project reports metrics designed for long-read phasing:

- **Number of phase sets:** a measure of block fragmentation (fewer/larger blocks generally indicate better long-range phasing).
- **Switch error rate:** the fraction of adjacent phased heterozygous variants where predicted phase flips relative to truth.
- **Phasing rate on shared heterozygous sites:** the fraction of shared heterozygous variants that are phased.
- **Effective phased recall:** an end-to-end metric combining variant set overlap and correctness of phasing on truth heterozygous variants (used to compare oracle vs called regimes consistently).

3. Functional Requirements (FRS-style)

This section specifies the functional requirements for the implemented benchmarking platform.

3.1 Product perspective and scope

The system is a reproducible benchmarking platform for evaluating WhatsHap in an ONT-like long-read phasing workflow. The pipeline targets the practical path:

Reference FASTA → reads FASTQ → alignment BAM → variants VCF → phased VCF → evaluation reports

The platform supports two evaluation regimes:

- **Oracle (ground-truth) VCF:** isolates phasing performance given the correct variant set.
- **Called VCF:** measures end-to-end performance including variant calling limitations.

3.2 Users and operating environment

Primary user: a developer/research user running controlled experiments and producing plots/tables for analysis and reporting.

Operating environment constraints:

- Python modules are executed via `python -m ... entrypoints`.
- The end-to-end pipeline requires external tools available on PATH:
 - minimap2, samtools, bcftools
- The pipeline runner checks tool availability at runtime (and fails early if missing).

3.3 Naming conventions and artifacts (prefix-based)

Most pipeline outputs are derived from a single `--prefix` argument used consistently by `benchmark.longread_pipeline_runner`. Given `--prefix output/exp1/runA`, the system produces artifacts following this convention:

- Reference: `output/exp1/runA.ref.fasta`, `output/exp1/runA.ref.meta.json`
- Truth: `output/exp1/runA.truth.vcf`, `output/exp1/runA.truth.vcf.gz` (+ index),
`output/exp1/runA.truth.meta.json`
- Oracle VCF: `output/exp1/runA.oracle.vcf`, `output/exp1/runA.oracle.vcf.gz` (+ index)
- Reads: `output/exp1/runA.reads.fastq`, `output/exp1/runA.reads.truth.tsv`, optional read meta JSON

- Alignment: `output/exp1/runA.bam (+ .bai)`
- Called variants: `output/exp1/runA.called.vcf.gz (+ .tbi)`
- WhatsHap outputs (called/oracle runs): `*.ws*.phased.vcf, *.ws*.eval.json, *.ws*.summary.json`
- Pipeline report: `output/exp1/runA.pipeline.json`

Aggregated experiments additionally produce:

- `aggregate.csv` under each experiment directory
- plots under `plots/` within the experiment directory

3.4 Functional requirements

FR-1 Reference generation

The system shall generate a synthetic reference genome FASTA with configurable realism presets and optional duplicated segments.

Implementation mapping:

- CLI: `python -m dataset.longread.reference`
- Called by: `python -m benchmark.longread_pipeline_runner`
- Parameters include:
 - `--ref-preset {plain,toy,realistic}`
 - `--ref-length`
 - duplication knobs: `--dup-segments, --dup-len, --dup-min-gap`
- Outputs: `*.ref.fasta, *.ref.meta.json`

FR-2 Ground-truth variant and haplotype generation

The system shall generate diploid ground-truth variants and two haplotype sequences, and export both a truth VCF and haplotype FASTAs.

Implementation mapping:

- CLI: `python -m dataset.longread.truth`
- Called by: `python -m benchmark.longread_pipeline_runner`
- Parameters include:
 - SNP knobs: `--num-snps, --het-rate`
 - optional indel knobs: `--num-indels, --indel-min-len, --indel-max-len, --indel-het-rate`
- Outputs:

- *.truth.vcf (+ .gz and index)
- *.hap1.fasta, *.hap2.fasta
- *.truth.meta.json
- *.oracle.vcf (+ .gz and index)

FR-3 ONT-like read simulation

The system shall simulate long reads from the diploid haplotypes, producing FASTQ with quality strings and read-truth metadata.

Implementation mapping:

- CLI: `python -m dataset.longread.readsim`
- Called by: `python -m benchmark.longread_pipeline_runner`
- Parameters include:
 - `--num-reads`, `--min-len`, `--max-len`
 - `--platform {ont,perfect}`, `--ont-profile {classic,q20}`
 - length model knobs: `--len-model {uniform,lognormal}`, `--ln-mean`, `--ln-sigma`
 - start/coverage model knobs: `--start-model {uniform,dropout}`, `--dropout-fraction`, `--dropout-block-len`
 - burst error knobs (passed via pipeline runner): `--burst-prob`, `--burst-count`, `--burst-len`, `--burst-mult`
- Outputs:
 - *.reads.fastq
 - *.reads.truth.tsv (read origin / truth mapping)
 - optional *.reads.meta.json if enabled by the generator

FR-4 Read alignment

The system shall align simulated reads to the reference and output a sorted, indexed BAM suitable for downstream variant calling and WhatsHap phasing.

Implementation mapping:

- Invoked by: `python -m benchmark.longread_pipeline_runner`
- Tools: `minimap2`, `samtools sort`, `samtools index`
- Parameter: `--map-preset` (default `map-ont`)
- Outputs: *.bam, *.bam.bai

FR-5 Variant calling

The system shall call variants from the aligned BAM against the reference and produce a called VCF.

Implementation mapping:

- Invoked by: `python -m benchmark.longread_pipeline_runner`
- Tools: `bcftools mpileup | bcftools call`
- Parameters include:
 - `--call-min-mapq` (`bcftools -q`)
 - `--call-min-baseq` (`bcftools -Q`)
- Outputs: `*,called.vcf.gz`, `*,called.vcf.gz.tbi`

FR-6 WhatsHap phasing (vendored integration)

The system shall phase variants using WhatsHap with BAM+VCF inputs and produce phased VCF output, using the vendored WhatsHap core for controlled benchmarking.

Implementation mapping:

- CLI endpoint: `python -m algorithms.cli.phase diploid-whats-bam`
- Driver: `algorithms/diploid/whatshap_bam_driver.py`
- Called by: `python -m benchmark.longread_pipeline_runner`
- Parameters include:
 - `--bam`, `--vcf`, `--output-vcf`, `--output-prefix`
 - `--max-coverage`, `--min-mapq`, `--min-baseq`
- Outputs:
 - `*,phased.vcf`
 - `*,summary.json` (instrumentation + counts)

FR-7 Phasing evaluation against truth

The system shall evaluate phased output against the truth VCF and output evaluation metrics focused on phasing correctness and fragmentation.

Implementation mapping:

- CLI: `python -m benchmark.vcf_phase_eval`
- Called by: `python -m benchmark.longread_pipeline_runner`
- Outputs: `*,eval.json` including:

- `phase_accuracy_blockflip`
- `switch_error_rate`
- `num_phase_sets`
- counts of shared/het/phased records

SNP-only policy for indels: When indels are enabled (`--num-indels > 0`), the system supports filtering to biallelic SNPs for phasing and/or evaluation:

- Pipeline runner flags: `--phase-snps-only`, `--eval-snps-only`
- These ensure evaluations remain valid when indel representations differ across truth/called/predicted VCFs.

FR-8 End-to-end reporting and aggregation

The system shall produce a single machine-readable report per run and provide aggregation utilities for experiment directories.

Implementation mapping:

- Run-level report: `*.pipeline.json` produced by `benchmark.longread_pipeline_runner`
- Aggregation:
 - `python -m benchmark.aggregate_pipeline_reports --root <dir> --out <csv>`
 - Used by `benchmark.experiment_driver`

The pipeline report shall record:

- parameter values used for the run
- file paths for produced artifacts
- calling precision/recall (when `--vcf-source` includes called)
- phasing evaluation metrics for oracle and/or called runs

FR-9 Experiment driver for systematic studies

The system shall provide a driver to execute predefined experiment suites across multiple seeds, aggregate results, and generate plots.

Implementation mapping:

- CLI: `python -m benchmark.experiment_driver --outdir ... --seeds ...`
- Selective execution: `--only <section1,section2,...>`
- Outputs per experiment section:
 - per-seed run artifacts
 - `aggregate.csv` and `plots/` for report-ready figures

3.5 Non-functional requirements (supporting the FYP goals)

- **Reproducibility:** runs must be seed-controlled and record parameters in `pipeline.json`.
- **Traceability:** all plots/tables must be derived from stored machine-readable outputs (`pipeline.json`, `aggregate.csv`).
- **Maintainability:** core evaluation and aggregation logic must be unit tested; legacy scripts/tests should be isolated from the main pipeline.
- **Portability:** code must run on a standard Linux environment with documented external tool dependencies.

4. System Design (SDD-style)

This section describes the design of the WhatsHap benchmarking platform implemented, focusing on architecture, component responsibilities, interfaces, and key design decisions that support reproducible end-to-end experiments.

4.1 Design objectives

The system is designed to:

1. **Benchmark WhatsHap in practical workflows**

Evaluate phasing in an end-to-end long-read pipeline (FASTA → FASTQ → BAM → VCF → phased VCF), not only on idealized matrices.

2. **Separate error sources**

Support both:

- **Oracle VCF** (ground-truth variants; isolates phasing behavior), and
- **Called VCF** (from bcftools; measures end-to-end caller + phaser performance).

3. **Be reproducible and traceable**

Every run is seed-controlled and produces a machine-readable report (`*.pipeline.json`) that records parameters, file paths, and metrics.

4. **Be modular and scriptable**

Each pipeline stage has a dedicated python `-m ...` entrypoint so it can be run independently or orchestrated by runners/drivers.

5. **Support realism knobs and systematic sweeps**

“Realism knobs” (duplications, dropout, bursts, indels) are parameterized and can be swept via an experiment driver.

4.2 High-level pipeline structure and tool mapping

The system is organized as a sequential pipeline with clearly defined stage boundaries. Each stage has (1) a single responsibility, (2) a primary implementation entrypoint in the repository, and (3) a well-defined input/output artifact contract.

4.2.1 Pipeline stages (conceptual view)

1. **Reference synthesis**

Create a reference FASTA with optional complexity/duplications.

2. Truth synthesis (diploid ground truth)

Create phased truth variants (truth VCF), derived haplotypes (hap1/hap2 FASTA), and an oracle VCF.

3. Read simulation (ONT-like)

Simulate reads from haplotypes, with configurable length models and error models.

4. Alignment + preprocessing

Align reads to the reference, producing a sorted/indexed BAM suitable for calling and phasing.

5. Variant calling

Produce a called VCF from the BAM and reference.

6. Phasing (WhatsHap)

Phase variants given BAM+VCF using the vendored WhatsHap core, producing phased VCF and summaries.

7. Evaluation + reporting

Compare predicted phasing to truth and produce JSON metrics; record run-level provenance in a pipeline report.

8. Experiment orchestration (optional layer)

Run multiple pipeline instances across seeds/knobs, aggregate reports into CSV, and generate plots.

This pipeline structure supports two evaluation regimes:

- **Oracle regime:** stages 1–4–6–7 using the oracle VCF to isolate phasing behavior.
- **Called regime:** stages 1–5–6–7 using the called VCF to measure end-to-end behavior.

4.2.2 Implementation mapping (codebase view)

Stage	Primary entrypoint in COMP4801_20	External tools	Key outputs
1	<code>dataset.longread.reference</code>	—	<code>*.ref.fasta</code> , <code>*.ref.meta.json</code>
2	<code>dataset.longread.truth</code>	<code>bcftools view/index</code> (bgzip+index)	<code>*.truth.vcf.gz</code> , <code>*.oracle.vcf.gz</code> , <code>*.hap1.fasta</code> , <code>*.hap2.fasta</code>
3	<code>dataset.longread.readsim</code>	—	<code>*.reads.fastq</code> , <code>*.reads.truth.tsv</code> , optional <code>*.reads.meta.json</code>
4	orchestrated by <code>benchmark.longread_pipeline_runner</code>	<code>minimap2</code> , <code>samtools sort/index</code>	<code>*.bam</code> , <code>*.bam.bai</code>

Stage	Primary entrypoint in COMP4801_20	External tools	Key outputs
5	orchestrated by <code>benchmark.longread_pipeline_runner</code>	<code>bcftools mpileup/call</code> , <code>*.called.vcf.gz</code> , <code>bcftools index</code> <code>*.called.vcf.gz.tbi</code>	
6	<code>algorithms.cli.phase_diploid-whats-bam</code> → <code>algorithms/diploid/whatshap_bam_driver.py</code>	(vendored) <code>whatshap_core</code>	<code>*.phased.vcf</code> , <code>*.summary.json</code>
7	<code>benchmark.vcf_phase_eval</code> + report writer in runner	—	<code>*.eval.json</code> , <code>*.pipeline.json</code>
8	<code>benchmark.experiment_driver</code> + <code>benchmark.aggregate_pipeline_reports</code>	—	<code>aggregate.csv</code> , <code>plots/*.png</code>

The single-run orchestrator `python -m benchmark.longread_pipeline_runner` is the canonical “glue” that runs stages 1–7 and writes `*.pipeline.json`. The experiment driver is a wrapper that repeatedly invokes the single-run orchestrator and then aggregates outputs.

4.2.3 Stage boundaries and responsibilities (quality view)

- **Data generation stages (1–3)** must be deterministic given a seed and must export metadata describing realism knobs applied.
- **Alignment/calling stages (4–5)** are treated as “tool-driven” steps; parameters are surfaced at the pipeline runner layer and recorded into `pipeline.json`.
- **Phasing stage (6)** must use the vendored WhatsHap core to avoid version drift; run summaries record module provenance.
- **Evaluation stage (7)** must be robust to caller differences; when indels are present, the pipeline supports SNP-only phasing/evaluation to avoid representation mismatches.
- **Experiment orchestration stage (8)** must not re-implement logic from stages 1–7; it only schedules runs and aggregates artifacts.

4.3 Component decomposition and responsibilities

4.3.1 Reference generator (`dataset.longread.reference`)

Responsibility: Generate a synthetic reference FASTA with configurable complexity and duplicated segments.

Outputs:

- `*.ref.fasta`
- `*.ref.meta.json` containing:

- regions (complex windows such as homopolymers/STR/GC windows)
- duplications (source/destination coordinates for copied segments)
- coord_note: **0-based half-open** intervals [start0, end0)

Key extension points:

- `--preset {plain,toy,realistic}`
 - `--dup-segments, --dup-len, --dup-min-gap`
-

4.3.2 Truth + haplotype generator (`dataset.longread.truth`)

Responsibility: Generate diploid ground truth variants and haplotypes.

Outputs:

- `*.truth.vcf` (+ `*.truth.vcf.gz` + index)
- `*.hap1.fasta, *.hap2.fasta`
- `*.truth.meta.json`
- Oracle VCF template (written by pipeline runner; see §4.4)

Notes:

- The truth VCF is **phased** when `--phased-truth` is used.
 - Indels can be introduced via `--num-indels` and indel length parameters.
 - If `--avoid-regions` is enabled, truth generation can avoid complex reference regions using `*.ref.meta.json`.
-

4.3.3 Long-read simulator (`dataset.longread.readsim`)

Responsibility: Simulate long reads from haplotypes and emit FASTQ with qualities and read-truth metadata.

Outputs:

- `*.reads.fastq` (sequence + ASCII qualities)
- `*.reads.truth.tsv` (read-level truth such as haplotype origin; used for debugging/validation)
- optional `*.reads.meta.json` (read simulation parameters/statistics)

Realism knobs currently supported by the runner:

- Platform: `--platform {ont,perfect}`

- ONT profile: `--ont-profile {classic,q20}`
 - Length distribution: `--len-model {uniform,lognormal}`, `--ln-mean`, `--ln-sigma`
 - Start/coverage model: `--start-model {uniform,dropout}`, plus dropout params
 - Correlated error bursts: `--burst-prob`, `--burst-count`, `--burst-len`, `--burst-mult`
-

4.3.4 Orchestration runner (`benchmark.longread_pipeline_runner`)

Responsibility: Execute the full pipeline for a single run and write a complete run report.

Workflow steps (as implemented):

1. Run `dataset.longread.reference`
2. Run `dataset.longread.truth`
3. Create and index truth VCF (`bcftools view/index`)
4. Create oracle VCF from truth (`_write_oracle_vcf`) and bgzip/index (optional)
5. Run `dataset.longread.readsim`
6. Align reads → BAM using `minimap2 | samtools sort`, then `samtools index`
7. Call variants using `bcftools mpileup | bcftools call` (optional)
8. Phase using `algorithms.cli.phase diploid-whats-bam` for oracle/called (optional)
9. Evaluate phasing using `benchmark.vcf_phase_eval`

Design details:

- Uses `_which_or_fail()` to ensure external tools exist on `PATH`.
- Uses `_piped()` to reliably run piped commands with error propagation.
- Uses prefix-based naming and writes a single report:
 - `*.pipeline.json` containing:
 - * `steps` (file paths)
 - * `params` (all run parameters)
 - * `callset` (calling precision/recall when applicable)
 - * `phasing_runs` (oracle/called subreports with derived metrics)
 - * `counts_raw` (record counts for sanity checks)

Indel-aware evaluation policy:

- When indels are present, the runner supports:
 - `--phase-snp-only`: filter phasing input VCF to biallelic SNPs
 - `--eval-snp-only`: filter truth VCF to biallelic SNPs for evaluation
 This prevents evaluation artifacts due to indel representation mismatches.
-

4.3.5 Unified phasing CLI (`algorithms.cli.phase`)

Responsibility: Provide a single CLI namespace for all phasing algorithms.

Relevant command for this project:

- `diploid-whats-bam`: WhatsHap-core phasing from BAM+VCF using vendored core.

This is the phasing entrypoint called by the pipeline runner.

4.3.6 WhatsHap BAM driver (`algorithms.diploid.whatsbam_driver`)

Responsibility: Implement WhatsHap-style phasing on BAM+VCF inputs using the **vendored** WhatsHap core.

Pipeline inside the driver:

1. Parse VCF (minimal parser) and keep **biallelic SNP records**; identify heterozygous sites.
2. Build WhatsHap ReadSet from BAM pileups at heterozygous SNP sites:
 - filters by `--min-mapq` and `--min-baseq`
 - only keeps reads covering ≥ 2 **variants** (required by WhatsHap read selection)
3. Perform read selection using `whatsbam.readselect.readselection` with `--max-coverage`.
4. Solve phasing using one of:
 - `core.PedigreeDPTable` (WhatsHap default-like solver), or
 - `core.HapChatCore` (optional)
5. Derive PS phase-set labels using connectivity of selected reads.
6. Write phased VCF by replacing GT and adding PS.
7. Emit auxiliary outputs:
 - `*.haplotypes.tsv`
 - `*.assignments.tsv` (placeholder assignments; consistent with project output format)
 - `*.summary.json` (counts + module provenance)

Vendored WhatsHap enforcement:

- `algorithms.vendor.whatsbam_vendor.import_whatsbam_vendor()` ensures `import whatsbam` resolves to `vendor/whatsbam_core`, preventing version drift and keeping benchmarking controlled.
-

4.3.7 Evaluation (`benchmark.vcf_phase_eval`)

Responsibility: Compare phased VCF to truth VCF and compute phasing metrics.

Outputs:

- `*.eval.json` containing:
 - record overlap counts (shared SNPs, shared hets, phased hets)
 - `num_phase_sets`
 - `phase_accuracy_blockflip`
 - switch error numerator/denominator and rate
 - per-phase-set breakdown (`phase_sets`)

Assumptions:

- Evaluation focuses on **biallelic SNPs**, and SNP-only filtering is used when indels are enabled.
-

4.3.8 Aggregation (`benchmark.aggregate_pipeline_reports`)

Responsibility: Convert many `*.pipeline.json` reports into a single `aggregate.csv` with standardized columns for plotting and analysis.

This is the “data contract” between experiment execution and plotting.

4.3.9 Experiment driver (`benchmark.experiment_driver`)

Responsibility: Run predefined experiment suites across multiple seeds, then aggregate and plot results.

Key design features:

- Runs the pipeline runner repeatedly with controlled argument sets.
 - Skips existing runs unless `--force` is used.
 - Automatically enables SNP-only phasing/evaluation when indels are configured.
 - Produces:
 - `aggregate.csv` per experiment directory
 - `plots/*.png` (error-bar plots across seeds)
 - optional decomposition CSVs (caller vs phaser bottlenecks)
-

4.4 Data contracts and file-level interfaces

The design relies on stable file formats exchanged between stages:

- **FASTA**: reference and haplotypes (`*.ref.fasta`, `*.hap1.fasta`, `*.hap2.fasta`)
- **FASTQ**: reads with qualities (`*.reads.fastq`)
- **BAM**: sorted + indexed alignments (`*.bam`, `*.bam.bai`)
- **VCF**:
 - truth (`*.truth.vcf.gz`), oracle (`*.oracle.vcf.gz`), called (`*.called.vcf.gz`)
 - phased outputs (`*.ws*.phased.vcf`)
- **JSON**:
 - `*.ref.meta.json` (regions + duplications; 0-based half-open coords)
 - `*.truth.meta.json` (truth generation parameters/summary)
 - `*.ws*.summary.json` (phasing run summary)
 - `*.ws*.eval.json` (phasing evaluation)
 - `*.pipeline.json` (single-run full report)

Prefix-based naming ensures that all artifacts from a run can be discovered from a single identifier.

4.5 Realism knobs as configurable stressors

The platform treats “realism knobs” as **configurable stressors**: each knob is a controlled perturbation that (a) approximates a real-world difficulty mode, and (b) is expected to measurably affect either calling metrics, phasing metrics, or both.

These knobs are intentionally:

- **parameterized** (CLI flags),
- **stage-local** (implemented in the stage where the phenomenon arises), and
- **fully recorded** in `*.pipeline.json` (so aggregated results are traceable).

4.5.1 Stressor catalogue and where it applies

Stressor (knob)	Pipeline stage	Why it matters	CLI / recorded fields
Reference complexity preset	Reference	Creates hard contexts (repeats, GC bias) affecting mapping and calling	<code>--ref-preset</code> ; <code>P.ref.meta.json</code> regions; <code>pipeline.json.params.ref_preset</code>
Duplicated segments	Reference	Introduces ambiguous mapping → lower call recall / fragmented phase sets	<code>--dup-segments</code> , <code>--dup-len</code> , <code>--dup-min-gap</code> ; <code>P.ref.meta.json.duplications</code>
Avoid complex regions for truth	Truth	Controls whether variants appear inside hard regions	<code>--avoid-regions</code> (uses <code>P.ref.meta.json</code>)
Truth indels	Truth	Indel representation mismatch affects both calling and evaluation if not controlled	<code>--num-indels</code> , <code>--indel-min-len</code> , <code>--indel-max-len</code> , <code>--indel-het-rate</code>
Read length distribution	Readsim	Affects effective coverage uniformity and phase connectivity	<code>--len-model</code> , <code>--ln-mean</code> , <code>--ln-sigma</code> ; <code>P.reads.meta.json</code>
Coverage dropout model	Readsim	Creates local low-coverage gaps → phase block fragmentation	<code>--start-model dropout</code> , <code>--dropout-fraction</code> , <code>--dropout-block-len</code>
Correlated error bursts	Readsim	Models read segments with elevated errors → call errors / read selection changes	<code>--burst-prob</code> , <code>--burst-count</code> , <code>--burst-len</code> , <code>--burst-mult</code>
SNP-only phasing/eval	Runner / Eval	Prevents indel encoding from dominating evaluation	<code>--phase-snp-only</code> , <code>--eval-snp-only</code>

4.5.2 Measurement expectations (what changes when knobs increase)

- **Caller-focused effects** (primarily in called regime):
 - `call_recall` decreases under duplications / dropout / bursts
 - `precision` may vary depending on caller thresholds and error severity
- **Phaser-focused effects** (oracle regime isolates these):

- `num_phase_sets` increases under dropout and duplications (fragmentation)
- switch error can increase when reads contain bursty errors and fewer reliable alleles survive filtering
- **End-to-end effects** (called regime):
 - “effective phased recall” typically decreases the most because it is limited by both calling overlap and phasing completeness

These expectations justify treating realism knobs as stressors that can be validated independently before running optimization experiments.

4.6 Key design decisions (rationale)

1. Prefix-based artifact naming

Simplifies orchestration, cleanup, and aggregation; avoids implicit state.

2. Oracle vs called regimes

Enables attribution of performance loss to calling vs phasing.

3. Vendored WhatsHap core

Prevents dependency drift and ensures controlled benchmarking; provenance is recorded in summary JSON.

4. SNP-only phasing/evaluation when indels are present

Avoids false failures due to indel representation differences across truth/called/phased VCFs.

5. Read filtering for WhatsHap compatibility

Reads covering fewer than 2 variants are excluded because WhatsHap read selection requires ≥ 2 covered variants.

6. Single-run JSON report as the source of truth

Aggregation and plotting rely on `*,pipeline.json`, not on parsing logs.

4.7 Extensibility

The design supports incremental realism and experimentation:

- **New realism knobs** can be added in `dataset.longread.reference/truth/readsim`, then surfaced via:
 - `benchmark.longread_pipeline_runner` (single-run support)
 - `benchmark.experiment_driver` (sweep support)
 - `benchmark.aggregate_pipeline_reports` (CSV column support)

- **Alternative phasers** can be added by:
 - implementing a new driver under `algorithms/`
 - registering a new subcommand in `algorithms.cli.phase`
 - adding calls and metrics extraction in the pipeline runner/driver

5. Implementation

This section describes how the system is implemented, focusing on the repository structure, key modules, the end-to-end runner, and the WhatsHap integration choices that enable controlled benchmarking.

5.1 Repository structure (implementation map)

The repository is organized around three concerns: data generation, phasing algorithms, and benchmarking/orchestration.

- `dataset/longread/`

Implements long-read pipeline data generation:

- `reference.py`: reference FASTA synthesis + complexity presets + duplication metadata
- `truth.py`: truth variants + haplotypes (truth VCF + hap1/hap2 FASTA; optional indels)
- `readsim.py`: ONT-like read simulation (FASTQ + read-truth TSV + meta JSON; length/dropout/bursts)

- `algorithms/`

Implements phasing algorithm CLIs and drivers:

- `cli/phase.py`: unified entrypoint `python -m algorithms.cli.phase`
- `diploid/whatshap_bam_driver.py`: WhatsHap phasing from BAM+VCF using vendored core
- `vendor/whatshap_vendor.py` (or equivalent): ensures imports resolve to vendored WhatsHap

- `benchmark/`

Implements end-to-end orchestration, evaluation, experiments, and aggregation:

- `longread_pipeline_runner.py`: single-run orchestrator producing `*.pipeline.json`
- `vcf_phase_eval.py`: phasing evaluation vs truth VCF
- `aggregate_pipeline_reports.py`: produces `aggregate.csv` from many `pipeline.json`
- `experiment_driver.py`: runs experiment suites across seeds, aggregates and plots

- `tests/`

Unit tests for evaluation/aggregation and specific realism implementations.

- `vendor/whatshap_core/`

Vendored WhatsHap core module (`whatshap`) with compiled extensions, used for controlled benchmarking.

This structure keeps the “single source of truth” for pipeline behavior in `benchmark.longread_pipeline_runner` and keeps experiments as wrappers around it.

5.2 End-to-end orchestration: `benchmark.longread_pipeline_runner`

The canonical workflow is implemented as a single-run pipeline runner:

- Entry point: `python -m benchmark.longread_pipeline_runner`
- Primary argument: `--prefix <path-prefix>`

The runner enforces two design contracts: 1. **Prefix-based artifact naming**: all produced files share the same prefix, enabling easy discovery. 2. **Machine-readable run report**: each run produces a `*.pipeline.json` capturing parameters, file outputs, and metrics.

5.2.1 Pipeline execution steps

For a run with prefix `P`, the runner performs the following steps:

1. Reference

- Executes: `python -m dataset.longread.reference -o P --length ... --seed ... --preset ... --dup-seg...`
- Produces: `P.ref.fasta`, `P.ref.meta.json`

2. Truth + haplotypes

- Executes: `python -m dataset.longread.truth --ref P.ref.fasta -o P --seed ... --num-snps ... --het-...`
- Produces: `P.truth.vcf`, `P.hap1.fasta`, `P.hap2.fasta`, `P.truth.meta.json`
- Compress/index truth VCF using `bcftools view/index` → `P.truth.vcf.gz`

3. Oracle VCF

- Generates an oracle VCF from truth and compress/indexes it:
 - `P.oracle.vcf.gz` (+ index)
- Oracle keeps the truth variant set and is used to isolate phasing behavior.

4. Read simulation

- Executes: `python -m dataset.longread.readsim --hap1 P.hap1.fasta --hap2 P.hap2.fasta -o P ...`
- Produces: `P.reads.fastq`, `P.reads.truth.tsv`, `P.reads.meta.json`

5. Alignment

- Executes:
 - `minimap2 -a -x <map_preset> P.ref.fasta P.reads.fastq | samtools sort -o P.bam`
 - `samtools index P.bam`
- Produces: `P.bam`, `P.bam.bai`

6. Variant calling (called regime)

- Executes:

- `bcftools mpileup -Ou -f P.ref.fasta -q <MAPQ> -Q <BASEQ> P.bam | bcftools call -mv -Oz -o P.c`
- `bcftools index -t P.called.vcf.gz`
- Produces: `P.called.vcf.gz`, `P.called.vcf.gz.tbi`

7. Phasing (oracle and/or called)

- Executes (for each chosen VCF input):
 - `python -m algorithms.cli.phase diploid-whats-bam --bam P.bam --vcf <VCF> --output-vcf <OUT> -`
- Produces:
 - phased VCF: `*.phased.vcf`
 - WhatsHap run summary: `*.summary.json`

8. Evaluation

- Executes:
 - `python -m benchmark.vcf_phase_eval --truth <truth> --pred <phased.vcf> --out <eval.json>`
- Produces: `*.eval.json`

9. Run report

- Writes: `P.pipeline.json` containing:
 - `params` (all effective parameters, including realism knobs and flags)
 - `steps` (file paths)
 - `callset` (calling precision/recall when called regime is enabled)
 - `phasing_runs` (oracle and/or called metrics + derived measures)

5.2.2 Toolchain validation and robustness

The runner performs early validation: - Ensures `minimap2`, `samtools`, and `bcftools` exist on `PATH` before executing. - Uses piped-command helpers to propagate failures from upstream tools. - Stores intermediate and final artifacts even when only partial regimes are enabled (oracle only, called only, or both).

5.3 WhatsHap integration: `diploid-whats-bam`

Phasing is implemented as a dedicated algorithm target:

- CLI: `python -m algorithms.cli.phase diploid-whats-bam`
- Driver: `algorithms/diploid/whatsbam_driver.py`

This driver is designed to benchmark WhatsHap while maintaining full control of: - input selection (which variants are phased), - read filtering (mapQ/baseQ thresholds), - read selection (max coverage), - output formatting (phased VCF + summary JSON).

5.3.1 Vendored WhatsHap core

To avoid version drift and ensure controlled benchmarking, the project uses a vendored WhatsHap core under `vendor/whatschap_core`. The code ensures that `import whatschap` resolves to the vendored module and records provenance in `*.summary.json`:

- `whatschap_module`
- `whatschap_core_module`
- `whatschap_readselect_module`

This provides traceability between reported results and the exact phasing implementation used.

5.3.2 Read filtering and selection

The driver applies filters to approximate practical usage: - discard reads below `--min-mapq` - discard allele observations below `--min-baseq` - discard reads that cover fewer than two variants (WhatsHap read selection expects ≥ 2 covered variants) - apply read selection with `whatschap.readselect.readselection` at `--max-coverage`

5.3.3 Phased VCF output

The driver writes a phased VCF by: - streaming the input VCF records, - replacing GT with phased genotypes for phased heterozygous sites, and - writing PS phase set tags to label blocks.

Homozygous genotypes do not require phasing and are generally preserved as unphased (0/0, 1/1) or left unchanged depending on configuration.

5.4 Evaluation implementation: `benchmark.vcf_phase_eval`

Evaluation compares predicted phasing to truth using biallelic SNP records and reports:

- shared record counts (total, SNP, heterozygous)
 - number of phased heterozygous records
 - number of phase sets (fragmentation)
 - phase accuracy under best block flip
 - switch error rate (switches / adjacency denominator)
 - per-phase-set breakdown
-

5.5 Indel-aware policy: SNP-only phasing/evaluation flags

When indels are introduced, representation differences between truth, called, and phased VCFs can break “exact match” comparisons. To keep evaluation meaningful, the platform supports:

- `--phase-snp-only`: filter the phasing input VCF to biallelic SNPs
- `--eval-snp-only`: filter the truth VCF to biallelic SNPs for evaluation

These flags are recorded in `*.pipeline.json` and surfaced into `aggregate.csv` so downstream plots always reflect whether SNP-only policy was applied.

5.6 Experiment automation and aggregation

5.6.1 Experiment driver

- Entry point: `python -m benchmark.experiment_driver`
- Runs multiple pipeline instances across seeds and parameter sets.
- Produces per-experiment:
 - run artifacts (`*.pipeline.json` per run),
 - `aggregate.csv`,
 - plots under `plots/`.

5.6.2 Aggregation contract

`benchmark.aggregate_pipeline_reports` reads each `*.pipeline.json` and outputs a standardized `aggregate.csv` containing: - configuration columns (including realism knobs and SNP-only flags) - callset metrics (precision/recall where applicable) - phasing metrics for oracle/called regimes - derived metrics (effective phased recall, shared het recall, etc.)

This CSV is the primary input for plotting and for generating tables in the report.

5.7 Realism knobs: where they are implemented and how they are surfaced

The platform’s realism features are not “one monolithic simulator.” Instead, realism is decomposed into stage-appropriate knobs, implemented where the phenomenon naturally arises:

1. **Reference-level stressors (`dataset.longread.reference`)**

- Presets: `--preset {plain,toy,realistic}`
- Duplications: `--dup-segments`, `--dup-len`, `--dup-min-gap`
- Export: `P.ref.meta.json` containing regions and duplications

2. Truth-level stressors (`dataset.longread.truth`)

- Indels: `--num-indels`, `--indel-min-len`, `--indel-max-len`, `--indel-het-rate`
- Placement control: `--avoid-regions` (requires `--ref-meta`)
- Export: `P.truth.meta.json` and a phased `P.truth.vcf(.gz)` when `--phased-truth` is used

3. Read-level stressors (`dataset.longread.readsim`)

- Length distribution: `--len-model {uniform,lognormal}`, `--ln-mean`, `--ln-sigma`
- Coverage dropout: `--start-model dropout`, `--dropout-fraction`, `--dropout-block-len`
- Correlated error bursts: `--burst-prob`, `--burst-count`, `--burst-len`, `--burst-mult`
- Export: `P.reads.meta.json` capturing the effective parameters used

4. Runner-level policy stressors (`benchmark.longread_pipeline_runner`)

- VCF selection: `--vcf-source {oracle,called,both}`
- Indel-safe evaluation: `--phase-snps-only`, `--eval-snps-only`
- Recorded in: `P.pipeline.json.params` and aggregated into `aggregate.csv`

This structure makes it possible to validate each realism knob independently (unit validation), and later construct controlled sweeps where only one stressor changes at a time.

6. Testing and Validation (STD-style)

This section describes the testing strategy, test environment assumptions, implemented test cases, and validation evidence for the benchmarking platform. The goal is to ensure correctness and reproducibility of (i) core data structures and metrics, (ii) WhatsHap integration, and (iii) the end-to-end pipeline orchestration and reporting.

6.1 Testing objectives

The test plan is designed to verify:

1. Correctness of core representations and I/O

- reads matrices and conversions (TSV/NPZ)
- adapter correctness when bridging project data to WhatsHap data structures

2. Correctness of phasing outputs

- phased haplotype TSV format
- phased VCF output formatting (GT and PS)
- use of **vendored WhatsHap core** (provenance checks)

3. Correctness of evaluation metrics

- block-flip-invariant accuracy
- switch error rate
- phase set counting

4. End-to-end reproducibility

- seed-controlled generation
- deterministic output contracts (prefix-based artifacts)
- stable reporting (`*.pipeline.json` → `aggregate.csv`)

6.2 Test levels and scope

The project uses a layered testing approach:

- **Unit tests**
 - Focus on small, deterministic components (I/O parsing, metric computations, adapters).
- **Integration tests**
 - Exercise CLIs and drivers with minimal synthetic inputs (NPZ/VCF mode).
 - Verify required outputs and sanity constraints.
- **System / end-to-end tests (smoke)**

- Use the experiment/pipeline runner to validate the real long-read workflow when the external toolchain is installed.

This separation is necessary because the long-read pipeline depends on external binaries (`minimap2`, `samtools`, `bcftools`) that may not be available in all test environments.

6.3 Test environment and dependencies

6.3.1 Python environment

- Python 3.11
- `pytest` for test execution

6.3.2 Vendored WhatsHap core

- Tests that invoke WhatsHap drivers require the vendored `vendor/whatshap_core` to be importable as `whatshap`.
- Where appropriate, tests skip when `whatshap.core` is not importable.

6.3.3 External toolchain (system tests)

System/end-to-end long-read runs require: `- minimap2 - samtools - bcftools`

These are treated as “system dependencies” and are validated at runtime by the pipeline runner.

6.4 Implemented unit tests (current repository)

The following unit-level tests are included under `tests/`:

- `tests/test_whatshap_adapter.py`
 - Validates compatibility between the project’s read representation and WhatsHap `ReadSet` requirements.
 - Ensures variant indexing and allele encoding match expected semantics.
- `tests/test_diploid_whatshap.py`
 - Minimal correctness test for the diploid WhatsHap NPZ-mode driver on a toy matrix.
 - Verifies output artifact creation (`haplotypes.tsv`, `assignments.tsv`, `summary.json`) and basic allele sanity.
- `tests/test_pedigreedptable_vcf_mode.py`
 - Regression test for VCF-mode phasing using the WhatsHap DP solver path.

- Asserts that heterozygous sites become phased (0|1 / 1|0) and share a consistent PS for a connected component.
- **tests/test_simulation_whatsnap_compat.py, tests/test_data_simulation_io.py**
 - Legacy / matrix-mode tests validating sparse/dense semantics and older simulation I/O.
 - These provide regression coverage for earlier components that remain in the codebase.

6.5 Implemented integration tests (current repository)

Integration tests execute real CLIs to validate routing and artifact contracts:

- **tests/test_vcf_mode_pipeline.py**
 - End-to-end regression on the matrix-mode pipeline:
 - * `dataset.simulate` → `algorithms.cli.phase diploid-whats` (VCF-mode) → output VCF/TSV checks
 - Verifies the phased VCF contains phased heterozygous records and that reconstructed haplotypes reach a conservative accuracy threshold.
- **tests/test_pipeline_whatsnap_cli.py**
 - Exercises the unified phasing CLI with `diploid-whats`.
 - Verifies the driver uses the **vendored** WhatsHap module by checking paths recorded in `summary.json`.
- **tests/test_pipeline_integration.py**
 - Exercises an older end-to-end flow (`simulate` → `convert` → `phase` → `evaluate accuracy JSON`) for regression coverage.

6.6 System / end-to-end smoke validation (long-read pipeline)

Because the long-read pipeline relies on external tools and is more expensive than unit tests, it is validated as a **smoke test** using the orchestrators:

- **Single-run orchestrator**
 - `python -m benchmark.longread_pipeline_runner --prefix ...`
 - Validates the full practical path: `reference` → `truth` → `read simulation` → `BAM` → `VCF` → `WhatsHap phasing` → `evaluation` → `pipeline.json`.
- **Experiment driver**
 - `python -m benchmark.experiment_driver --outdir ... --seeds ...`
 - Validates:
 - * repeated runs across seeds

- * correct aggregation from *.pipeline.json to aggregate.csv
- * plot generation under plots/

Pass criteria for smoke validation - All expected artifacts for the selected regime exist (oracle/called/both). - *.pipeline.json is produced and parseable. - *.eval.json contains required keys (e.g., switch_error_rate, num_phase_sets). - Aggregation produces a non-empty aggregate.csv with expected columns.

6.7 Validation for realism knobs (correctness criteria)

Realism knobs are validated using both metadata checks and metric-based sanity checks:

- **Duplications (reference stressor)**

- *.ref.meta.json contains a duplications list with valid, non-overlapping coordinate pairs.
- Expected effect: decreased calling recall and/or increased phase set fragmentation under higher duplication counts.

- **Coverage dropout (read start model)**

- *.reads.meta.json records dropout parameters.
- Expected effect: more phase sets (fragmentation) and reduced shared heterozygous recall in called regime.

- **Correlated error bursts**

- Read simulator parameters are recorded in *.reads.meta.json.
- Expected effect: reduced calling recall or increased unphased rate due to fewer reliable allele observations.

- **Truth indels**

- Indel introduction is recorded in *.truth.meta.json.
- To keep evaluation meaningful, SNP-only phasing/evaluation is used (--phase-snp-only, --eval-snp-only) when indels are enabled.
- Expected effect: if SNP-only policy is applied, SNP phasing metrics should remain stable across indel severity sweeps (within noise), indicating indels are not corrupting SNP evaluation.

6.8 Coverage limitations and recommended additional tests

Current limitations - The repository's automated pytest suite primarily covers matrix-mode and driver correctness; the long-read toolchain workflow is validated via smoke runs (requires external binaries).

Recommended additions (future work / if time permits) 1. **Long-read runner smoke test (skipped if tools missing)** - Run benchmark.longread_pipeline_runner with very small parameters (oracle-only) and assert pipeline.json schema.

2. Schema/contract unit tests

- Validate that `pipeline.json` contains required fields and that aggregation produces mandatory columns.

3. Reference meta integrity tests

- Deterministically generate a reference with duplications and assert coordinates are consistent (0-based half-open, within bounds, non-overlapping given min-gap).

4. SNP-only policy regression

- Enable indels and assert that enabling `eval_snps_only` prevents spuriously low block-flip accuracy caused by indel representation mismatches.

These additions would strengthen automated coverage for the long-read realism pipeline without requiring large experiment runtimes.

7. Software Configuration Management Plan (SCMP-style)

This section describes how source code, experimental configurations, and generated artifacts are managed to ensure traceability and reproducibility of results reported in this project.

7.1 Configuration items (CIs)

The primary configuration items are:

1. Source code repository

- All Python modules under `dataset/`, `algorithms/`, and `benchmark/`
- Vended phasing core under `vendor/whatshap_core/`
- Documentation under `docs/` and project report drafts under `report/` (if applicable)

2. Experiment definitions

- Experiment suites and parameter grids defined in `benchmark/experiment_driver.py`
- Single-run parameterization defined by `benchmark/longread_pipeline_runner.py`

3. Run artifacts and reports

- Prefix-scoped run artifacts (FASTA/FASTQ/BAM/VCF)
- Machine-readable run reports: `*.pipeline.json`
- Aggregated datasets: `aggregate.csv`
- Plots derived from aggregates: `plots/*.png`

7.2 Version control strategy

The project uses Git to track changes to all code and documentation.

Recommended branch roles (reflecting actual workflow in this project): - `main`: stable baseline used to generate report results and final experiments - `feature/*`: isolated development of new realism knobs, evaluation features, or drivers - `cleanup/*`: refactoring, removal of obsolete scripts/tests, documentation reorganization

A change is considered “ready for main” once: - unit/integration tests pass (`pytest`) - the end-to-end smoke run is successful (when the external toolchain is available) - the change does not break the report artifact contracts (`*.pipeline.json` and `aggregate.csv` schemas)

7.3 Change control and traceability

To ensure results are traceable to code versions:

- Each experiment run is associated with:

- a commit hash (recorded manually in experiment logs, or optionally stored in `pipeline.json.params.git_commit`)
 - a deterministic seed
 - a complete parameter record in `*.pipeline.json`
- All plots and tables included in the report must be derivable from:
 - the `*.pipeline.json` files produced by the runner, aggregated into
 - `aggregate.csv` via `benchmark.aggregate_pipeline_reports`

This design avoids dependence on ad-hoc terminal logs and makes the report reproducible.

7.4 Artifact naming and storage policy

Artifacts are generated using prefix-based naming to guarantee consistency and simplify cleanup.

For a run with prefix `P`: - `P.ref.fasta`, `P.truth.vcf.gz`, `P.reads.fastq`, `P.bam`, `P.called.vcf.gz` - WhatsHap outputs: `P.ws*.phased.vcf`, `P.ws*.summary.json`, `P.ws*.eval.json` - Run report: `P.pipeline.json`

Experiment directories are structured as: - `<outdir>/<experiment_section>/...` containing per-seed runs and final `aggregate.csv + plots/`.

Artifacts are treated as: - **source-of-truth**: `*.pipeline.json` (and `*.eval.json`, `*.summary.json`) - **derived**: `aggregate.csv`, plots

7.5 Baseline and release management

Key “release” points in the project correspond to: - stable versions used for report figures and tables - tagged commits (optional) such as `v1-report-baseline`

A recommended approach: - tag the commit used to generate final report experiments - store experiment outputs in a dedicated `output/<tag>/` directory or archive them as a zip for submission

7.6 Backup and recovery

To avoid data loss: - code and docs are backed up via the remote Git repository - final experiment outputs used in the report should be archived (zip) and stored separately from transient debugging outputs

8. Software Quality Assurance Plan (SQAP-style)

This section describes the quality assurance activities used to ensure that the platform is correct, maintainable, and suitable for generating report-grade experimental results.

8.1 Quality goals

The system targets the following quality properties:

- **Correctness:** Metrics, phasing outputs, and run reports reflect the true pipeline behavior.
- **Reproducibility:** Runs are seed-controlled and configurations are recorded in `*.pipeline.json`.
- **Traceability:** Report plots/tables are derived from stored machine-readable artifacts.
- **Maintainability:** The codebase is modular (stage-based) and supports incremental addition of realism knobs and experiments.
- **Robustness:** Failures in external tools are detected early and surfaced clearly.

8.2 Standards and conventions

- Code follows a consistent CLI-driven structure using `python -m <module>` entrypoints.
- Outputs follow prefix-based naming conventions, enabling deterministic file discovery.
- Module-level documentation is maintained under `docs/` and updated when new knobs/drivers are introduced.

8.3 Verification activities

The following verification activities are used:

1. Unit and integration tests (pytest)

- Validate metric computation, adapter correctness, and CLI routing.
- Provide regression coverage for core evaluation logic.

2. System smoke validation

- Validates the full long-read workflow through:
 - `benchmark.longread_pipeline_runner` (single-run)
 - `benchmark.experiment_driver` (multi-run)
- Confirms that expected artifacts exist and that JSON/CSV contracts are satisfied.

3. Realism knob validation

- Each realism knob is validated using:

- metadata integrity checks (`*.ref.meta.json`, `*.reads.meta.json`, `*.truth.meta.json`)
- sanity expectations on metrics (e.g., duplication tends to reduce calling recall and/or increase fragmentation)

8.4 Defect handling and regression prevention

- Defects are addressed by:
 - reproducing with a fixed seed and saving the failing run's `*.pipeline.json`
 - adding a regression test when feasible (unit test or small integration test)
- A change is not considered complete until it:
 - passes the relevant tests, and
 - preserves the reporting contract (pipeline JSON schema and aggregation columns)

8.5 Documentation QA

Documentation is kept consistent with the codebase by: - referencing runnable CLIs (`python -m ...`) rather than informal scripts - documenting new flags when new realism knobs are added - ensuring docs describe both oracle and called evaluation regimes

8.6 Limitations

- Full system validation requires external binaries (`minimap2/samtools/bcftools`), so CI-style testing may only cover unit/integration tests.
- Some realism properties are approximations; validation focuses on correctness of implementation and expected qualitative effects rather than perfect biological fidelity.

9. Project Management Plan (PMP-style)

This section summarizes the project's deliverables, milestones, and risk management approach, focusing on the work completed toward the final report and experimental study.

9.1 Deliverables

The final deliverables of this project are:

1. Benchmarking platform implementation

- End-to-end pipeline:
 - reference → truth/oracle → read simulation → alignment → calling → phasing → evaluation → report
- Vendedored WhatsHap integration for controlled benchmarking

2. Realism knob implementations

- Reference-level stressors (e.g., duplications / complexity presets)
- Read-level stressors (dropout, length models, correlated error bursts)
- Truth-level stressors (indels) with SNP-only policy for meaningful evaluation

3. Experiment infrastructure

- Experiment driver for running systematic studies across seeds and knobs
- Aggregation and plotting utilities producing report-ready figures and CSVs

4. Validation artifacts

- Unit/integration tests and smoke-run outputs
- *.pipeline.json reports and aggregated aggregate.csv datasets used by the report

5. Final report

- Documentation of requirements, design, implementation, testing, experiments, and conclusions

9.2 Work plan and milestones (final-phase focus)

A practical final-phase work plan:

- **M1: Platform stabilization**
 - ensure smoke validation passes on target environment
 - confirm pipeline.json and aggregate.csv contain required columns for report tables/plots
- **M2: Realism knob completion and validation**

- validate each realism knob with targeted runs and sanity checks
- ensure SNP-only policy is consistently applied when indels are enabled

- **M3: Experiment execution**

- run the planned experiment suites (oracle vs called regimes)
- aggregate results and generate plots

- **M4: Report completion**

- integrate experiment plots/tables into the report
- finalize discussion of optimization opportunities and limitations

9.3 Risks and mitigations

Risk	Impact	Mitigation
External toolchain not available / inconsistent versions	prevents end-to-end experiments	document dependencies; fail early in runner; test on target machine
Evaluation artifacts due to indel representation mismatch	misleading accuracy/switch metrics	SNP-only phasing/evaluation flags; validate with indel sweeps
Time constraints in final phase	reduced experiment coverage	prioritize experiments that directly support report claims; use smaller parameter sweeps
Overfitting to synthetic data	reduced realism	use realism knobs to approximate failure modes; discuss threats to validity
Codebase complexity	harder to maintain	cleanup branch; isolate legacy scripts/tests; keep <code>pipeline.json</code> schema stable

9.4 Tracking and reporting

Progress is tracked through: - Git commits and branch history for code changes - prefix-scoped outputs and `pipeline.json` reports for experiment evidence - aggregated CSVs and plots for report integration

10. Experimental Evaluation

10.1 Experimental methodology and common setup

10.1.1 Environment and toolchain

- OS / CPU: macOS / Apple M2 Max
- Python version: 3.12.12
- minimap2 version: 2.30-r1287
- samtools version: 1.23
- bcftools version: 1.23
- Vended WhatsHap core path: `vendor/whatschap_core/whatschap/core.<platform-specific extension>`

10.1.2 Pipeline regimes

- **Oracle regime:** phase using `*.oracle.vcf.gz` (phaser-limited)
- **Called regime:** phase using `*.called.vcf.gz` (end-to-end)
- **Indel policy:** when indels are enabled, use SNP-only phasing/evaluation (`phase_snps_only`, `eval_snps_only`)

For this project, a custom adaptor layer was used to construct the `ReadSet` consumed by the vended WhatsHap core. This was necessary to support the study’s experimental requirements, including oracle vs called benchmarking, unified JSON provenance, and controlled SNP-only phasing/evaluation under indel-containing truth sets. The adaptor is therefore part of the project’s benchmarking infrastructure rather than a direct reproduction of the standard production-facing WhatsHap front-end.

10.1.3 Fixed parameters (baseline constants)

Unless stated otherwise in a subsection, experiments use the baseline defaults defined by the experiment driver. These parameters control each stage of the end-to-end pipeline (reference/truth generation → read simulation → alignment → calling → phasing → evaluation).

Reference generation - `ref_length`: Length of the synthetic reference genome (bp). Larger references reduce boundary effects and allow realism stressors (e.g., duplications) to be placed without overlap. - **`ref_preset`:** Reference “complexity preset” (e.g., plain vs realistic). Presets control the presence of hard regions (e.g., repetitive or biased regions) and are recorded in `*.ref.meta.json`. - **`dup_segments`:** Number of duplicated segments introduced into the reference. Higher values create more ambiguous mapping, typically reducing calling recall and increasing phasing fragmentation. - **`dup_len`:** Length (bp) of each duplicated segment copied from one locus to another. - **`dup_min_gap`:** Minimum separation (bp) enforced between duplicated segments/placements to avoid trivial overlaps.

Truth generation - **num_snps**: Number of SNP sites inserted into the truth set. Controls variant density and the potential number of heterozygous loci for phasing. - **het_rate**: Fraction of truth SNPs that are heterozygous (e.g., 0/1). WhatsHap only needs to phase heterozygous sites; homozygous sites do not contribute phasing difficulty. - **phased_truth**: Whether the truth VCF contains phased GT (e.g., 0|1). This is required for switch-error-based evaluation. - **random_phase**: Whether truth phase orientation is randomly assigned per site (used to avoid biasing truth haplotypes). - **Indel parameters (only when enabled)**: - **num_indels**: Number of indel variants introduced into truth. - **indel_min_len, indel_max_len**: Indel length range (bp). Larger indels can increase representation differences between truth/called sets. - **indel_het_rate**: Fraction of indels that are heterozygous.

Read simulation - **platform**: Read simulation platform (e.g., ont). Determines error profile assumptions. - **ont_profile**: Preset for substitution/insertion/deletion rates and base-quality characteristics (e.g., q20). Used to approximate practical ONT error rates. - **num_reads**: Number of reads simulated. This acts as a proxy for sequencing depth in this synthetic setting. - **min_len, max_len**: Bounds on read lengths (bp). Affects overlap connectivity (and therefore phasing block sizes). - **hap1_frac**: Fraction of reads sampled from haplotype 1 (vs haplotype 2). Typically 0.5 for balanced coverage. - **Length model parameters**: - **len_model**: Distribution used to draw read lengths (e.g., uniform or lognormal). - **ln_mean, ln_sigma**: Parameters of the lognormal model (used when `len_model=lognormal`). - **Start/coverage model parameters**: - **start_model**: How read start positions are sampled (e.g., uniform vs dropout). - **dropout_fraction**: Fraction of the reference affected by coverage dropout (low/near-zero coverage windows). - **dropout_block_len**: Typical length of dropout blocks (bp), controlling the size of connectivity gaps. - **Correlated error burst parameters**: - **burst_prob**: Probability a read contains one or more “bursts” of elevated error. - **burst_count**: Number of bursts injected into a read (often fixed or bounded). - **burst_len**: Length of each burst (bp). - **burst_mult**: Multiplier applied to base error rates within bursts (higher → more severe).

Alignment and calling thresholds - **map_preset**: Minimapp2 mapping preset (e.g., map-ont). Chosen to match ONT alignment behavior. - **call_min_mapq**: Minimum mapping quality for reads to be included in variant calling (`bcftools mpileup -q`). Higher thresholds reduce ambiguous mappings but may reduce coverage. - **call_min_baseq**: Minimum base quality for bases to contribute to variant calling (`bcftools mpileup -Q`). Higher thresholds reduce noisy evidence but may reduce sensitivity.

Phasing parameters (WhatsHap integration) - **vcf_source**: Which VCF to use for phasing: - **oracle**: truth-derived variant set (isolates phasing), - **called**: called variant set (end-to-end), - **both**: run both for attribution. - **max_coverage**: WhatsHap read-selection cap (maximum effective coverage retained). Larger values retain more reads (potentially better connectivity) but increase compute. - **min_mapq**: Minimum mapping quality for reads to be used in phasing (driver filter). Controls how aggressively ambiguous alignments are excluded. - **min_baseq**: Minimum base quality for allele observations used in phasing (driver filter). Controls how aggressively noisy allele calls are excluded. - **Indel-safe evaluation policy flags**: - **phase_snps_only**: Filter *input to phasing* to biallelic SNPs only. - **eval_snps_only**: Filter *truth used for evaluation* to biallelic SNPs only. These flags prevent indel representation differences from corrupting SNP phasing evaluation.

10.1.4 Metrics reported (from `aggregate.csv`)

Metrics are computed per run from `*.pipeline.json` and `*.eval.json`, then aggregated across seeds into `aggregate.csv`. We report both **oracle** (phaser-focused) and **called** (end-to-end) metrics to attribute performance loss to variant calling vs phasing.

Calling quality (called regime only) - `call_precision`: Fraction of called SNPs that match truth SNP sites (low precision indicates false positives).

Intuition: “How many called variants are correct?” - **`call_recall`**: Fraction of truth SNPs recovered by the caller (low recall indicates missed variants).

Intuition: “How many true variants were found?” - **`truth_snps`**: Number of SNP records in the truth set (after any SNP-only filtering). - **`called_snps`**: Number of SNP records produced by the caller. - **`shared_snps`**: Number of SNP records present in both truth and called sets (the overlap the phaser can actually work with in called regime).

Phasing coverage / completeness - `*_shared_het_recall` (called): Fraction of truth heterozygous SNPs that appear in the phasing input overlap (i.e., how many true hets survive calling and exact matching).

This captures the “caller bottleneck” for phasing. - **`*_phasing_rate_shared_het` (called)**: Fraction of shared heterozygous SNPs that end up phased (i.e., GT becomes `0|1` or `1|0`).

This captures the “phaser completeness” *given* available het sites. - **`*_effective_phased_recall` (oracle/called)**: Fraction of truth heterozygous SNPs that are both (a) present in the evaluation set and (b) phased correctly (under best block flip).

This is the main “end-to-end usable phasing” metric.

Phasing correctness - `*_phase_accuracy` (oracle/called): Accuracy of phased heterozygous genotypes under an optimal global flip per phase set.

Reason: haplotypes are label-symmetric (swap hap1/hap2 yields equivalent solution), so evaluation must be flip-invariant. - **`*_switch_error` (oracle/called)**: Switch error rate, computed as `#switches / #adjacent_het_pairs_compared`.

Interpretation: how often the predicted phase relationship between adjacent heterozygous variants is inconsistent with truth.

Switch error is often the most informative correctness metric in long-read phasing.

Phase block fragmentation - `*_num_phase_sets` (oracle/called): Number of distinct phase sets (PS) in the phased VCF.

Lower is better (fewer blocks means longer haplotype continuity), but extremely low values can also occur if very few het sites are phased. - **Phase set size distribution (from eval JSON)**: Per-phase-set sizes and best-flip matches.

Used to interpret whether fragmentation is driven by coverage gaps (dropout), ambiguous mapping (duplications), or filtering thresholds.

Runtime / efficiency - `time_total_sec`: Total pipeline runtime per run (if recorded).

Used in optimization sweeps to quantify trade-offs between accuracy/continuity and compute.

Oracle vs called interpretation - Oracle metrics represent “phaser-limited” performance (variants are correct). -

Called metrics represent “end-to-end” performance (caller + phaser). - The gap between oracle and called effective phased recall indicates how much performance is lost due to variant calling overlap rather than phasing itself.

10.2 Baseline: depth scaling (coverage proxy via num_reads)

Aim

Establish baseline performance curves as sequencing depth increases, and attribute performance limitations by comparing the oracle (phaser-limited) regime against the called (end-to-end) regime.

Setup

- Varied: num_reads $\in \{50, 100, 200, 400\}$ (3 seeds each)
- Fixed: ONT-like simulation profile (q20), reference length 80 kb, 800 truth SNPs (het rate 0.8), read length range 2–6 kb, calling thresholds (mapQ/baseQ), and phasing thresholds (max_coverage, min_mapq, min_baseq) as specified in §10.1.

Figures to include

- **Figure 10.2.1 — Variant calling recall vs sequencing depth.**
- **Figure 10.2.2 — Oracle effective phased recall vs sequencing depth (phasing-only upper bound).**
- **Figure 10.2.3 — Called effective phased recall vs sequencing depth (end-to-end performance).**
- **Figure 10.2.4 — Oracle number of phase sets vs sequencing depth (block fragmentation, phasing-only).**
- **Figure 10.2.5 — Called number of phase sets vs sequencing depth (block fragmentation, end-to-end).**
- **Figure 10.2.6 — Called switch error rate vs sequencing depth.**

Figure 10.2.1 — Variant calling recall vs sequencing depth.

Calling recall is computed as `shared_snps / truth_snps`, where `shared_snps` are SNP records present in both the truth VCF and the called VCF (exact-position matching in the pipeline’s callset summary). As read depth increases (`num_reads`), recall increases sharply, indicating that *variant discovery* is the main limiter at low coverage. Points show mean over seeds; error bars show standard deviation.

Figure 10.2.2 — Oracle effective phased recall vs sequencing depth (phasing-only upper bound).

Oracle effective phased recall measures WhatsHap’s phasing performance when the input variants are “oracle” (truth-derived SNP sites), removing variant calling errors from the pipeline. The curve rises with depth and approaches a high plateau, representing the phasing-only capability under the simulated ONT Q20 error model. Points show mean over seeds; error bars show standard deviation.

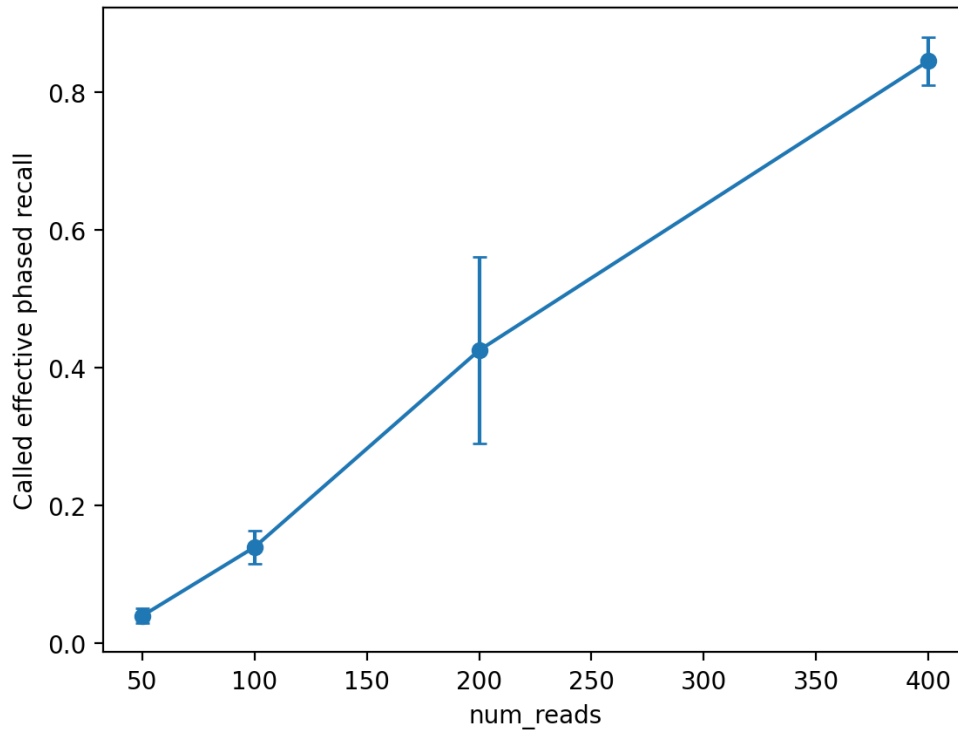


Figure 10.2.3 — Called effective phased recall vs sequencing depth (end-to-end performance). Called effective phased recall measures the fraction of truth heterozygous SNPs that end up *both present in the called set and correctly phased* (after best block flip). This curve is substantially lower than the oracle curve at low depth, demonstrating that variant calling recall (and callset composition) dominates end-to-end phasing performance early on. Points show mean over seeds; error bars show standard deviation.

Figure 10.2.4 — Oracle number of phase sets vs sequencing depth (block fragmentation, phasing-only). The number of phase sets (PS blocks) reflects *fragmentation*: more phase sets means shorter blocks / less connectivity. Under oracle variants, increasing depth reduces fragmentation as reads bridge more heterozygous sites. Points show mean over seeds; error bars show standard deviation.

Figure 10.2.5 — Called number of phase sets vs sequencing depth (block fragmentation, end-to-end). The phase-set count using called variants reflects fragmentation under end-to-end conditions. Unlike the oracle curve, this may be non-monotonic at moderate depths because the called VCF changes with depth (different sites called and filtered), affecting graph connectivity. Points show mean over seeds; error bars show standard deviation.

Figure 10.2.6 — Called switch error rate vs sequencing depth. Switch error rate is computed over adjacent phased heterozygous SNP pairs within phase sets (after choosing the best global flip per phase set). The mean generally decreases as depth increases, but can show variance at intermediate depths due to small denominators and changing callsets. Error bars show standard deviation; interpret values as bounded to $[0, 1]$ even if $\text{mean} \pm \text{std}$ extends below 0.

Results summary (means over seeds)

Calling (called regime): - `call_recall`: 0.224 (50) → 0.399 (100) → 0.662 (200) → 0.877 (400) - `call_precision`: 1.000 at all depths in this synthetic setup (no SNP false positives observed)

Oracle phasing (phaser-limited): - `oracle_effective_phased_recall`: 0.513 → 0.750 → 0.921 → 0.969 - `oracle_switch_error`: near 0 across all depths - `oracle_num_phase_sets`: 10.7 → 5.7 → 1.7 → 1.0

Called phasing (end-to-end): - `called_effective_phased_recall`: 0.040 → 0.140 → 0.426 → 0.845 - `called_shared_het_recall`: 0.129 → 0.281 → 0.590 → 0.854 - `called_phasing_rate_shared_het`: 0.355 → 0.517 → 0.808 → 0.991 - `called_switch_error`: high variance at 100–200 reads (outlier seeds), near 0 at 400 reads - `called_num_phase_sets`: 5.0 → 7.7 → 3.0 → 1.0 (non-monotonic due to changing number of phased het sites)

Taken together, these decomposition terms show that most loss in called effective phased recall arises from reduced shared heterozygous site recall rather than from poor phase accuracy, indicating that variant recovery and connectivity are the dominant bottlenecks in the baseline setting.

Key observations

- O1 (Calling is depth-limited): Calling recall increases strongly with `num_reads` (0.224 → 0.877), indicating the caller is primarily sensitivity-limited at low/moderate depth. Precision remains 1.0 in this controlled setup, so errors manifest mainly as missed sites rather than spurious sites.
- O2 (WhatsHap is accurate given correct sites): In the oracle regime, switch error is ~0 and phase accuracy is ~1 even at low depth; the main limitation at 50–100 reads is incompleteness (many het sites remain unphased due to connectivity gaps).
- O3 (Residual high-depth loss remains caller-limited): At 400 reads, called phasing is nearly perfect on shared heterozygous sites (phasing rate ≈ 0.99 , accuracy ≈ 1.0). The remaining gap between oracle and called effective phased recall is therefore dominated by missing heterozygous sites in the called set (`called_shared_het_recall` ≈ 0.85).
- O4 (Mid-depth instability in called regime): At 100–200 reads, the called regime shows occasional high switch error in individual seeds. This likely reflects both seed sensitivity and small denominators in switch-error calculation when the called VCF contains relatively few informative heterozygous sites. This motivates using more seeds in the final pass.

Takeaway

The baseline depth sweep confirms that the pipeline behaves sensibly: calling recall increases with depth; oracle phasing becomes both accurate and contiguous with increasing depth; and end-to-end (called) phasing improves sharply but remains limited primarily by the overlap of correctly called heterozygous variants. At sufficiently high depth (400 reads in this setup), WhatsHap phasing is essentially correct on available het sites, implying that further end-to-end gains would require improvements in variant calling overlap rather than the phasing algorithm itself.

10.3 Single-knob study: duplicated regions (reference stressor)

Aim

Measure how duplicated reference segments affect end-to-end phasing by increasing mapping ambiguity, and determine whether the main impact is on variant recovery, phasing continuity, or phasing correctness.

Setup

- Varied: `dup_segments` $\in \{0, 1, 3, 5\}$ (3 seeds each by default; 5 seeds recommended if runtime allows)
- Fixed:
 - `dup_len` = 3000
 - `dup_min_gap` = 500
 - baseline depth fixed at `num_reads` = 200
 - ONT-like simulation profile (`ont_profile` = q20)
 - reference length 80 kb
 - 800 truth SNPs (`het_rate` = 0.8)
 - read length range 2–6 kb
 - `start_model` = uniform
 - no dropout (`dropout_fraction` = 0.0)
 - no correlated bursts (`burst_prob` = 0.0)
 - no truth indels (`num_indels` = 0)
 - phasing input source `vcf_source` = both

This experiment isolates duplicated sequence as a single realism stressor while keeping all other baseline conditions unchanged.

Figures to include

- **Figure 10.3.1 — Variant calling recall vs duplicated-region count.**
- **Figure 10.3.2 — Called effective phased recall vs duplicated-region count.**
- **Figure 10.3.3 — Called switch error rate vs duplicated-region count.**
- **Figure 10.3.4 — Oracle effective phased recall vs duplicated-region count.**
- **Figure 10.3.5 — Called number of phase sets vs duplicated-region count.**

Figure 10.3.1 — Variant calling recall vs duplicated-region count.

Calling recall remains nearly unchanged across duplication levels, indicating that the duplicated-region stressor does not substantially reduce SNP site recovery under the current alignment and calling settings.

Figure 10.3.2 — Called effective phased recall vs duplicated-region count.

End-to-end phasing remains broadly stable at low-to-moderate duplication levels, but declines at the highest duplication setting. This indicates that duplicated regions only become a substantial stressor in the called regime once ambiguity is sufficiently strong.

Figure 10.3.3 — Called switch error rate vs duplicated-region count.

Switch error remains modest at low-to-moderate duplication levels but increases markedly at the highest duplication setting, suggesting that duplicated regions can destabilize local phase relationships even when overall call recall remains similar.

Figure 10.3.4 — Oracle effective phased recall vs duplicated-region count.

Oracle phasing performance remains high and stable across duplication levels, showing that WhatsHap itself is robust when correct variant sites are available and that the main duplication effect emerges in the end-to-end called regime.

Figure 10.3.5 — Called number of phase sets vs duplicated-region count.

Phase-set count does not increase with duplication in this experiment and therefore should not be interpreted in isolation. At the highest duplication level, fewer phase sets coincide with worse phasing accuracy and higher switch error, indicating that phase-set count alone is not a sufficient measure of performance here.

Results summary (means over seeds)

- `call_recall`: 0.663 (0 dup) → 0.671 (1 dup) → 0.661 (3 dup) → 0.666 (5 dup)
- `call_precision`: 1.000 at all duplication levels in this synthetic setup
- `called_effective_phased_recall`: 0.436 → 0.473 → 0.479 → 0.372
- `called_shared_het_recall`: 0.590 → 0.599 → 0.586 → 0.593
- `called_phasing_rate_shared_het`: 0.819 → 0.836 → 0.848 → 0.778
- `called_phase_accuracy`: 0.890 → 0.920 → 0.960 → 0.802
- `called_switch_error`: 0.122 → 0.073 → 0.040 → 0.209
- `called_num_phase_sets`: 3.4 → 3.4 → 3.4 → 2.6
- `oracle_effective_phased_recall`: 0.925 → 0.932 → 0.924 → 0.918
- `oracle_switch_error`: near 0 across all duplication levels
- `oracle_num_phase_sets`: 1.8 → 1.8 → 1.2 → 1.8

Taken together, these results show that duplicated regions are a relatively mild stressor under the current parameterization. The strongest degradation appears only at the highest duplication level (`dup_segments = 5`), and is driven mainly by reduced called-regime phasing completeness and phase accuracy rather than by reduced callset overlap.

Key observations

- O1 (Calling recall is largely unaffected by duplication in this setup): Across `dup_segments` $\in \{0, 1, 3, 5\}$, calling recall remains nearly flat (0.663 – 0.671) and call precision stays at 1.0 . This indicates that the current duplication stressor does not substantially reduce SNP site recovery under the baseline alignment/calling settings.
- O2 (Oracle phasing remains stable): Oracle effective phased recall remains high and nearly unchanged (0.918 – 0.932), with near-zero oracle switch error throughout. This shows that when correct variant sites are supplied, WhatsHap itself remains robust to the duplicated-region stressor at these settings.
- O3 (Called-regime degradation appears mainly at the highest duplication level): At `dup_segments` = 5, called effective phased recall drops from 0.479 to 0.372 , while called switch error rises from 0.040 to 0.209 . However, `called_shared_het_recall` remains approximately constant (~ 0.59), indicating that the degradation is not primarily due to loss of shared heterozygous sites.
- O4 (Main loss is in phasing completeness/correctness on surviving sites): The highest-duplication condition reduces `called_phasing_rate_shared_het` ($0.848 \rightarrow 0.778$) and `called_phase_accuracy` ($0.960 \rightarrow 0.802$). This suggests that duplicated regions mainly weaken the consistency of phasing evidence in the called regime rather than simply reducing callset overlap.
- O5 (Phase-set count is not the main signal here): `called_num_phase_sets` does not increase with duplication, and is actually lower at `dup_segments` = 5. In this experiment, lower phase-set count does not imply better phasing, because it occurs alongside lower effective phased recall and higher switch error. This indicates that phase-set count should be interpreted together with completeness and correctness metrics rather than in isolation.

Takeaway

Under the current parameterization, duplicated regions are a relatively mild stressor for variant recovery and for oracle phasing. Their main impact appears only at the highest duplication level, where end-to-end (called) phasing becomes less complete and less accurate despite largely unchanged callset overlap. This suggests that duplicated regions primarily affect the quality and stability of phasing evidence in the called regime, rather than acting as a strong caller-limited bottleneck. In other words, the duplication stressor in this setup is better characterized as a called-regime phasing/evidence-quality stressor than as a pure site-recovery stressor.

10.4 Single-knob study: coverage dropout (read start model)

Aim

Evaluate how localized coverage dropout affects phase-block continuity and effective phased recall, and determine whether the main impact is on variant recovery, phasing completeness, or both.

Setup

- Varied: `dropout_fraction` $\in \{0.00, 0.05, 0.10, 0.20\}$
- Fixed:
 - `dropout_block_len` = 1000
 - baseline depth fixed at `num_reads` = 200
 - ONT-like simulation profile (`ont_profile` = q20)
 - reference length 80 kb
 - 800 truth SNPs (`het_rate` = 0.8)
 - read length range 2–6 kb
 - `start_model` = dropout
 - no duplications (`dup_segments` = 0)
 - no correlated bursts (`burst_prob` = 0.0)
 - no truth indels (`num_indels` = 0)
 - phasing input source `vcf_source` = both

This experiment isolates coverage dropout as a single realism stressor while keeping the remaining baseline conditions unchanged.

Figures to include

- **Figure 10.4.1 — Oracle number of phase sets vs dropout fraction.**
- **Figure 10.4.2 — Called number of phase sets vs dropout fraction.**
- **Figure 10.4.3 — Oracle effective phased recall vs dropout fraction.**
- **Figure 10.4.4 — Called effective phased recall vs dropout fraction.**
- **Figure 10.4.5 — Variant calling recall vs dropout fraction.**

Figure 10.4.1 — Oracle number of phase sets vs dropout fraction. The number of oracle phase sets increases strongly with dropout fraction, showing that localized coverage gaps fragment haplotype blocks even when correct variant sites are supplied to the phaser.

Figure 10.4.2 — Called number of phase sets vs dropout fraction. End-to-end phase fragmentation increases at low-to-moderate dropout levels, but phase-set count alone becomes harder to interpret at severe dropout because many heterozygous sites are no longer called or phased.

Figure 10.4.3 — Oracle effective phased recall vs dropout fraction. Oracle effective phased recall decreases as dropout increases, indicating that coverage gaps directly reduce phasing completeness by breaking read connectivity between heterozygous sites.

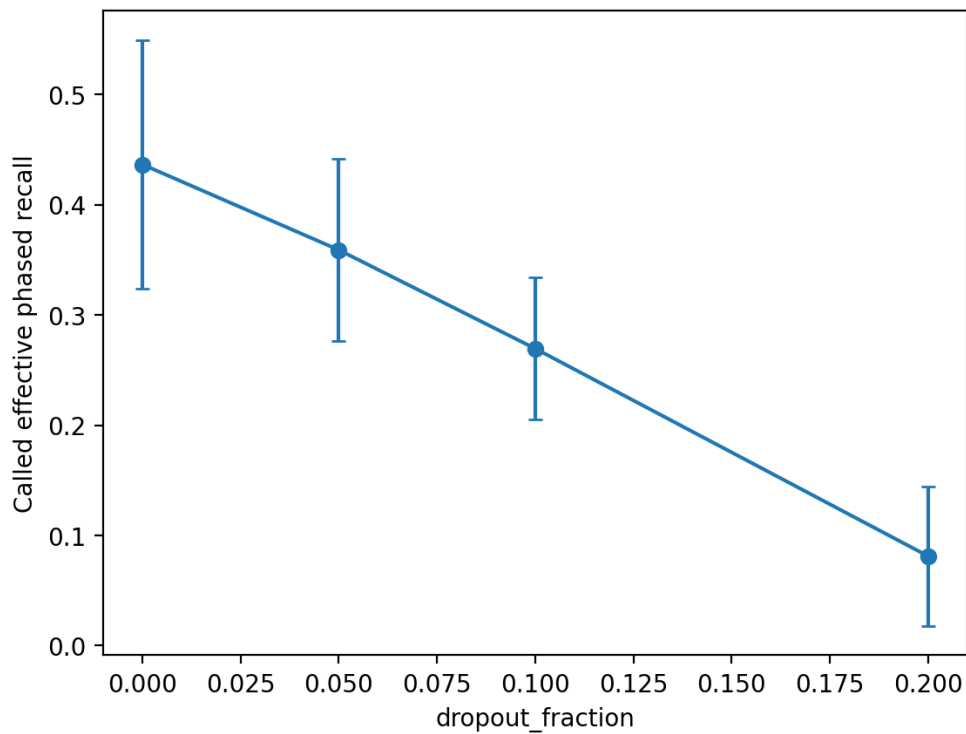


Figure 10.4.4 — Called effective phased recall vs dropout fraction. Called effective phased recall declines much more steeply than oracle effective phased recall, showing that dropout affects both phasing connectivity and, at stronger dropout levels, variant recovery in the called regime.

Figure 10.4.5 — Variant calling recall vs dropout fraction.

Calling recall decreases gradually at moderate dropout and then sharply at the strongest dropout level, indicating that severe low-coverage regions substantially reduce the recovery of true SNP sites.

Results summary (means over seeds)

- `call_recall`: 0.663 (0.00) → 0.635 (0.05) → 0.605 (0.10) → 0.432 (0.20)
- `call_precision`: 1.000 at all dropout levels in this synthetic setup
- `oracle_effective_phased_recall`: 0.925 → 0.865 → 0.857 → 0.637
- `oracle_switch_error`: near 0 across all dropout levels
- `oracle_num_phase_sets`: 1.8 → 2.8 → 3.2 → 10.0
- `called_effective_phased_recall`: 0.436 → 0.359 → 0.269 → 0.081
- `called_shared_het_recall`: 0.590 → 0.565 → 0.529 → 0.362
- `called_phasing_rate_shared_het`: 0.819 → 0.704 → 0.514 → 0.239
- `called_phase_accuracy`: 0.890 → 0.894 → 0.981 → 0.922

- `called_switch_error`: 0.122 → 0.072 → 0.006 → 0.089
- `called_num_phase_sets`: 3.4 → 5.0 → 6.2 → 4.0

Decomposition of called effective phased recall shows that coverage dropout first increases loss through unphased shared heterozygous sites (fragmentation / reduced connectivity), and at the strongest dropout level also causes a substantial increase in loss from missing called sites.

Key observations

- O1 (Coverage dropout strongly increases fragmentation): In the oracle regime, the number of phase sets rises from 1.8 at `dropout_fraction = 0.00` to 10.0 at 0.20, while oracle switch error remains near zero throughout. This shows that dropout primarily breaks phase-block continuity rather than causing incorrect phase assignments.
- O2 (Dropout directly reduces phasing completeness even with correct variant sites): Oracle effective phased recall declines from 0.925 to 0.637 as dropout increases. Because oracle shared-site recall remains fixed by construction, this reduction is attributable to weaker read connectivity and more unphased heterozygous sites.
- O3 (End-to-end performance degrades more strongly than oracle performance): Called effective phased recall falls from 0.436 to 0.081, a steeper decline than in the oracle regime. This indicates that dropout harms end-to-end phasing through both connectivity loss and reduced overlap of correctly called heterozygous variants.
- O4 (Moderate dropout is mainly a connectivity / completeness problem): From `dropout_fraction = 0.00` to 0.10, the main worsening in the called regime is a drop in `called_phasing_rate_shared_het` (0.819 → 0.514) together with increased fragmentation. This indicates that moderate dropout primarily prevents surviving heterozygous sites from being phased into long connected blocks.
- O5 (Severe dropout becomes a mixed calling + phasing bottleneck): At `dropout_fraction = 0.20`, `call_recall` drops sharply to 0.432 and `called_shared_het_recall` falls to 0.362, while `called_phasing_rate_shared_het` also falls to 0.239. This shows that strong dropout not only fragments phase blocks but also substantially reduces variant recovery in the called regime.
- O6 (Switch error is not the main signal in this study): Although called switch error does not increase monotonically, this should not be interpreted as improved phasing at moderate dropout. As dropout increases, fewer heterozygous sites remain phased, making switch error less informative than completeness and fragmentation metrics.

Takeaway

Coverage dropout is a strong fragmentation-dominated stressor in this pipeline. Its primary effect is to create low-coverage gaps that prevent reads from connecting heterozygous sites into long phase blocks, which reduces

phasing completeness even in the oracle regime. In the called regime, this effect is amplified, and at severe dropout levels the stressor becomes mixed: both phase connectivity and variant recovery deteriorate substantially. Overall, coverage dropout is one of the clearest examples in this study of a realism knob that directly disrupts haplotype continuity.

10.5 Single-knob study: correlated error bursts

Aim

Assess how localized high-error segments within reads affect variant calling and phasing stability, and determine whether the main impact is on callset recovery, phasing correctness, or phasing completeness.

Setup

- Varied: `burst_prob` $\in \{0.0, 0.3, 0.6\}$
- Fixed:
 - `burst_count` = 2
 - `burst_len` = 300
 - `burst_mult` = 8.0
 - baseline depth fixed at `num_reads` = 200
 - ONT-like simulation profile (`ont_profile` = q20)
 - reference length 80 kb
 - 800 truth SNPs (`het_rate` = 0.8)
 - read length range 2–6 kb
 - `start_model` = uniform
 - no duplications (`dup_segments` = 0)
 - no coverage dropout (`dropout_fraction` = 0.0)
 - no truth indels (`num_indels` = 0)
 - phasing input source `vcf_source` = both

This experiment isolates correlated error bursts as a single realism stressor while keeping the remaining baseline conditions unchanged.

Figures to include

- **Figure 10.5.1 — Called switch error rate vs burst probability.**
- **Figure 10.5.2 — Called effective phased recall vs burst probability.**
- **Figure 10.5.3 — Variant calling recall vs burst probability.**

- **Figure 10.5.4 — Called number of phase sets vs burst probability.**
- **Figure 10.5.5 — Oracle effective phased recall vs burst probability.**

Figure 10.5.1 — Called switch error rate vs burst probability.

Called switch error shows a modest increase at the intermediate burst setting, suggesting that correlated high-error segments can destabilize local phase relationships in the end-to-end called regime. However, the effect is not monotonic across the burst sweep.

Figure 10.5.2 — Called effective phased recall vs burst probability.

End-to-end phased recall decreases at `burst_prob = 0.3` but rebounds at `0.6`, indicating that the current burst stressor produces a weak and somewhat noisy effect rather than a strong monotonic degradation trend.

Figure 10.5.3 — Variant calling recall vs burst probability.

Calling recall remains broadly stable across burst settings, indicating that correlated error bursts do not substantially reduce SNP site recovery under the current parameterization.

Figure 10.5.4 — Called number of phase sets vs burst probability.

Phase-set count does not increase with burst probability and therefore is not the main signal in this study. Under the current settings, correlated error bursts appear to affect phasing correctness more than fragmentation.

Figure 10.5.5 — Oracle effective phased recall vs burst probability.

Oracle phasing performance remains high and stable across burst settings, showing that WhatsHap itself is robust when correct variant sites are supplied and that the observed burst effect arises mainly in the end-to-end called regime.

Results summary (means over seeds)

- `call_recall`: 0.663 (`burst_prob = 0.0`) → 0.681 (`0.3`) → 0.676 (`0.6`)
- `call_precision`: 1.000 at all burst levels in this synthetic setup
- `oracle_effective_phased_recall`: 0.925 → 0.926 → 0.921
- `oracle_switch_error`: near 0 across all burst levels
- `oracle_num_phase_sets`: 1.8 → 1.0 → 1.6
- `called_effective_phased_recall`: 0.436 → 0.369 → 0.455
- `called_shared_het_recall`: 0.590 → 0.611 → 0.608
- `called_phasing_rate_shared_het`: 0.819 → 0.790 → 0.837
- `called_phase_accuracy`: 0.890 → 0.766 → 0.887
- `called_switch_error`: 0.122 → 0.142 → 0.108
- `called_num_phase_sets`: 3.4 → 2.0 → 2.6

Decomposition of called effective phased recall shows that the main additional loss at `burst_prob = 0.3` comes from switch / phase error rather than from reduced shared-site recall. However, this effect is not monotonic, because the `burst_prob = 0.6` condition rebounds close to baseline on most metrics.

Key observations

- O1 (Calling recall is largely unaffected by bursty errors in this setup): Across `burst_prob` $\in \{0.0, 0.3, 0.6\}$, calling recall remains nearly flat (0.663–0.681) and call precision stays at 1.0. This indicates that the current burst stressor does not substantially reduce SNP site recovery under the baseline alignment and calling settings.
- O2 (Oracle phasing remains stable): Oracle effective phased recall stays near 0.92–0.93, oracle switch error remains near zero, and oracle phase-set counts stay low. This shows that WhatsHap itself is robust to the current burst stressor when correct variant sites are supplied.
- O3 (The main visible effect is a dip in called-regime phasing correctness at intermediate burst probability): At `burst_prob` = 0.3, called effective phased recall drops from 0.436 to 0.369, while called phase accuracy decreases from 0.890 to 0.766 and called switch error increases from 0.122 to 0.142. This indicates that bursty errors can reduce the stability of phasing decisions in the end-to-end called regime.
- O4 (The burst effect is not monotonic under the current parameterization): At `burst_prob` = 0.6, called effective phased recall rebounds to 0.455, while called phase accuracy and switch error return close to baseline. This suggests that the current burst sweep produces a weak and somewhat noisy signal rather than a strong monotonic degradation trend.
- O5 (The main additional loss at `burst_prob` = 0.3 comes from correctness rather than site recovery): Decomposition shows that the largest extra loss term at `burst_prob` = 0.3 is switch / phase error, whereas `called_shared_het_recall` remains stable (~ 0.61). This suggests that correlated error bursts mainly affect the quality of phasing evidence on surviving called sites rather than substantially reducing callset overlap.
- O6 (Fragmentation is not the dominant signal in this study): Called phase-set counts do not increase with burst probability and are lower than baseline in both burst settings. Therefore, correlated error bursts in the current setup are better characterized as a weak phasing-correctness stressor than as a fragmentation stressor.

Takeaway

Under the current parameterization, correlated error bursts are a relatively weak and noisy stressor in this pipeline. They do not substantially reduce calling recall or oracle phasing performance, indicating that neither variant recovery nor phasing with correct sites is strongly disrupted. The main visible effect is a reduction in called-regime phasing correctness at intermediate burst probability, suggesting that structured local errors can sometimes destabilize end-to-end phasing decisions. However, because this effect is not monotonic across the burst sweep, the present results should be interpreted as preliminary rather than as evidence of a strong burst-driven failure mode.

10.6 Single-knob study: read length model

Aim

Compare uniform and lognormal read length distributions to assess how read-length variability affects phasing connectivity, fragmentation, and end-to-end phased recall.

Setup

- Varied: `len_model` $\in$ {uniform, lognormal}
- Fixed:
 - baseline depth fixed at `num_reads` = 200
 - ONT-like simulation profile (`ont_profile` = q20)
 - reference length 80 kb
 - 800 truth SNPs (`het_rate` = 0.8)
 - read length range 2–6 kb for the uniform baseline
 - lognormal parameters fixed at `ln_mean` = 8.3, `ln_sigma` = 0.6
 - `start_model` = uniform
 - no duplications (`dup_segments` = 0)
 - no coverage dropout (`dropout_fraction` = 0.0)
 - no correlated bursts (`burst_prob` = 0.0)
 - no truth indels (`num_indels` = 0)
 - phasing input source `vcf_source` = both

This experiment isolates the read length distribution as a single realism stressor while keeping the remaining baseline conditions unchanged.

Figures to include

- **Figure 10.6.1 — Oracle number of phase sets by read length model.**
- **Figure 10.6.2 — Called number of phase sets by read length model.**
- **Figure 10.6.3 — Oracle effective phased recall by read length model.**
- **Figure 10.6.4 — Called effective phased recall by read length model.**
- **Figure 10.6.5 — Variant calling recall by read length model.**

Figure 10.6.1 — Oracle number of phase sets by read length model.

The lognormal read length model reduces the number of oracle phase sets relative to the uniform baseline, indicating improved long-range connectivity between heterozygous sites when correct variant sites are supplied.

Figure 10.6.2 — Called number of phase sets by read length model.

The lognormal read length model substantially reduces called phase-set count, showing that it improves phase-

block continuity under end-to-end conditions. However, this continuity gain does not translate into a large increase in effective phased recall.

Figure 10.6.3 — Oracle effective phased recall by read length model.

Oracle effective phased recall is slightly higher under the lognormal model, suggesting that the alternative read length distribution modestly improves phasing completeness when correct variant sites are available.

Figure 10.6.4 — Called effective phased recall by read length model.

Called effective phased recall changes only slightly between the uniform and lognormal models, indicating that improved connectivity and overlap are largely offset by somewhat noisier called-regime phasing correctness.

Figure 10.6.5 — Variant calling recall by read length model.

Calling recall is modestly higher under the lognormal read length model, indicating that the alternative read length distribution slightly improves recovery of true SNP sites in the called regime.

Results summary (means over seeds)

- `call_recall`: 0.663 (uniform) → 0.701 (lognormal)
- `call_precision`: 1.000 under both read length models in this synthetic setup
- `oracle_effective_phased_recall`: 0.925 → 0.931
- `oracle_switch_error`: near 0 under both models
- `oracle_num_phase_sets`: 1.8 → 1.4
- `called_effective_phased_recall`: 0.436 → 0.443
- `called_shared_het_recall`: 0.590 → 0.633
- `called_phasing_rate_shared_het`: 0.819 → 0.810
- `called_phase_accuracy`: 0.890 → 0.855
- `called_switch_error`: 0.122 → 0.165
- `called_num_phase_sets`: 3.4 → 1.6

Taken together, these results show that the lognormal read length model improves callset overlap and phase-block continuity, but these gains translate into only a very small increase in called effective phased recall because called-regime phasing correctness becomes slightly worse and more variable across seeds.

Key observations

- O1 (Lognormal read lengths improve callset overlap): Compared with the uniform baseline, the lognormal model increases `call_recall` (0.663 → 0.701) and `called_shared_het_recall` (0.590 → 0.633). This suggests that the alternative read length distribution provides slightly stronger variant-support coverage under the current settings.
- O2 (Lognormal read lengths improve phase-block continuity): Oracle phase-set count decreases from 1.8 to 1.4, and called phase-set count decreases more strongly from 3.4 to 1.6. Oracle effective phased recall

also increases slightly ($0.925 \rightarrow 0.931$). Together, these results indicate improved long-range connectivity between heterozygous sites under the lognormal read length model.

- O3 (End-to-end improvement is small despite better connectivity): Although overlap and continuity improve, called effective phased recall changes only slightly ($0.436 \rightarrow 0.443$). This indicates that better block connectivity alone is not sufficient to produce a large end-to-end gain in this setting.
- O4 (Completeness gains are offset by noisier called-regime correctness): Under the lognormal model, `called_phasing_rate_shared_het` decreases slightly ($0.819 \rightarrow 0.810$), `called_phase_accuracy` decreases ($0.890 \rightarrow 0.855$), and `called_switch_error` increases ($0.122 \rightarrow 0.165$). This suggests that while the lognormal distribution helps connect more sites, the resulting called-regime phasing is somewhat less stable across seeds.
- O5 (The read length model is therefore a mixed, not clearly beneficial, knob): The current comparison shows that read-length distribution can improve continuity-related metrics without producing a correspondingly strong improvement in end-to-end usable phasing output. This makes the read length model informative for mechanism analysis, but not yet a clear optimization lever under the present parameterization.

Takeaway

Under the current parameterization, the read length model is a mixed stressor rather than a clear optimization knob. The lognormal model improves callset overlap and phase-block continuity, indicating better long-range connectivity, but these benefits are largely offset by slightly worse and noisier called-regime phasing correctness. As a result, end-to-end effective phased recall changes only marginally. This suggests that changing the read length distribution alone is not sufficient to deliver a strong practical improvement in this pipeline, although it does reveal an important trade-off between connectivity and phasing stability.

10.7 Truth indels and SNP-only policy (evaluation robustness)

Aim

Show that introducing indels into the truth set does not invalidate SNP phasing evaluation when SNP-only phasing and evaluation are enforced. This section verifies that the benchmarking framework remains robust to indel-containing truth without conflating SNP phasing performance with indel representation mismatches.

Setup

- Varied: `num_indels` $\in \{0, 80, 200\}$
- Fixed:

- `indel_min_len = 1`
- `indel_max_len = 5`
- `indel_het_rate = 0.5`
- baseline depth fixed at `num_reads = 200`
- ONT-like simulation profile (`ont_profile = q20`)
- reference length 80 kb
- 800 truth SNPs (`het_rate = 0.8`)
- read length range 2–6 kb
- `start_model = uniform`
- no duplications (`dup_segments = 0`)
- no coverage dropout (`dropout_fraction = 0.0`)
- no correlated bursts (`burst_prob = 0.0`)
- phasing input source `vcf_source = both`
- SNP-only safeguards enabled automatically when indels are present:
 - * `phase_snps_only = true`
 - * `eval_snps_only = true`

This experiment isolates the effect of adding indels to truth generation while preserving SNP-only phasing and evaluation comparability.

Figures to include

- **Figure 10.7.1 — Oracle effective phased recall vs number of truth indels.**
- **Figure 10.7.2 — Called effective phased recall vs number of truth indels.**
- **Figure 10.7.3 — Variant calling recall vs number of truth indels.**
- **Figure 10.7.4 — Called number of phase sets vs number of truth indels.**

Figure 10.7.1 — Oracle effective phased recall vs number of truth indels.

Oracle SNP-phasing performance remains essentially unchanged as truth indels are introduced, showing that SNP-only phasing and evaluation preserve a stable phasing-only benchmark in indel-containing truth sets.

Figure 10.7.2 — Called effective phased recall vs number of truth indels.

Called SNP-phasing performance remains broadly stable across indel settings under SNP-only evaluation, indicating that truth indels do not invalidate end-to-end SNP phasing assessment when the appropriate safeguards are enabled.

Figure 10.7.3 — Variant calling recall vs number of truth indels.

Calling recall changes only modestly across indel settings, suggesting that the presence of truth indels has limited indirect impact on SNP recovery under the current pipeline and SNP-only evaluation policy.

Figure 10.7.4 — Called number of phase sets vs number of truth indels.

Called phase-set count varies across indel settings but does not show evidence of catastrophic fragmentation or

evaluation instability, supporting the conclusion that SNP-only safeguards preserve interpretable SNP phasing metrics in indel-containing truth sets.

Results summary (means over seeds)

- `call_recall`: 0.663 (`num_indels = 0`) → 0.691 (80) → 0.654 (200)
- `call_precision`: 1.000 → 0.999 → 0.997
- `oracle_effective_phased_recall`: 0.925 → 0.925 → 0.921
- `oracle_switch_error`: near 0 across all indel settings
- `oracle_num_phase_sets`: 1.8 → 1.2 → 1.8
- `called_effective_phased_recall`: 0.436 → 0.573 → 0.493
- `called_shared_het_recall`: 0.590 → 0.623 → 0.577
- `called_phasing_rate_shared_het`: 0.819 → 0.919 → 0.856
- `called_phase_accuracy`: 0.890 → 0.999 → 1.000
- `called_switch_error`: 0.122 → 0.001 → 0.000
- `called_num_phase_sets`: 3.4 → 2.4 → 4.0

Taken together, these results show that introducing truth indels does not destabilize SNP phasing evaluation when `phase_snps_only = true` and `eval_snps_only = true` are enabled. Oracle SNP-phasing metrics remain essentially unchanged, and called SNP metrics remain within a comparable range across indel settings.

Key observations

- O1 (Oracle SNP-phasing remains stable under truth indels): Oracle effective phased recall stays essentially unchanged (0.925 → 0.925 → 0.921), oracle switch error remains near zero, and oracle phase-set counts remain low. This shows that adding indels to truth generation does not corrupt SNP-only phasing evaluation when SNP-only safeguards are enabled.
- O2 (Called SNP-phasing does not collapse in the presence of indels): Called effective phased recall remains within a similar overall range across `num_indels` ∈ {0, 80, 200}, and calling recall stays relatively stable (0.663–0.691, then 0.654 at the strongest indel setting). This indicates that the benchmarking pipeline continues to produce interpretable SNP-level results even when truth indels are present.
- O3 (The main result is evaluation robustness, not a strong indel-driven degradation trend): The 80-indel condition performs slightly better than the no-indel baseline on several called metrics, including called effective phased recall and switch error. This should not be interpreted as evidence that indels improve SNP phasing; rather, it suggests that under SNP-only filtering the remaining differences are dominated by ordinary stochastic variation and indirect alignment/calling effects rather than by evaluation artifacts.
- O4 (Even stronger indel conditions remain manageable under SNP-only policy): At `num_indels = 200`, oracle metrics remain near baseline and called effective phased recall remains above the no-indel baseline,

while called switch error is effectively zero. This supports the claim that the SNP-only policy successfully prevents indel representation differences from invalidating SNP phasing evaluation.

- O5 (SNP-only safeguards are therefore justified methodologically): Overall, the indel suite supports the use of `phase_snps_only` and `eval_snps_only` as robust safeguards whenever truth indels are present, because they preserve stable and interpretable SNP phasing metrics without obvious representation-induced distortion.

Takeaway

The SNP-only policy successfully preserves interpretable SNP phasing evaluation in the presence of truth indels. Under the current settings, introducing indels into truth generation does not materially destabilize oracle or called SNP-level phasing metrics, indicating that indel representation mismatch has been effectively controlled. This validates the use of `phase_snps_only = true` and `eval_snps_only = true` as an evaluation safeguard for indel-containing truth sets in the remainder of the study.

10.8 Interaction study: duplications × dropout

Aim

Test whether duplicated regions and coverage dropout compound non-linearly, and determine whether the resulting weakness is mainly caller-limited, connectivity-limited, or mixed.

Setup

Small 2×2 interaction grid: - `dup_segments` ∈ {0, 5} - `dropout_fraction` ∈ {0.0, 0.1}

Fixed: - `dropout_block_len` = 1000 - `dup_len` = 3000 - `dup_min_gap` = 500 - baseline depth fixed at `num_reads` = 200 - ONT-like simulation profile (`ont_profile` = q20) - reference length 80 kb - 800 truth SNPs (`het_rate` = 0.8) - read length range 2–6 kb - `start_model` = dropout - no correlated bursts (`burst_prob` = 0.0) - no truth indels (`num_indels` = 0) - phasing input source `vcf_source` = both

This experiment tests whether two realistic stressors that were individually interpretable in earlier sections combine into a more severe end-to-end phasing failure mode.

Figures to include

- **Figure 10.8.1 — Variant calling recall across duplication × dropout conditions.**
- **Figure 10.8.2 — Called effective phased recall across duplication × dropout conditions.**
- **Figure 10.8.3 — Called number of phase sets across duplication × dropout conditions.**

- **Figure 10.8.4 — Called switch error across duplication × dropout conditions.** (secondary; interpret with caution)

Figure 10.8.1 — Variant calling recall across duplication × dropout conditions.

Calling recall is only mildly affected by duplication alone, but decreases further when duplication is combined with coverage dropout. This shows that ambiguous mapping compounds the loss of callable sites under low-coverage conditions.

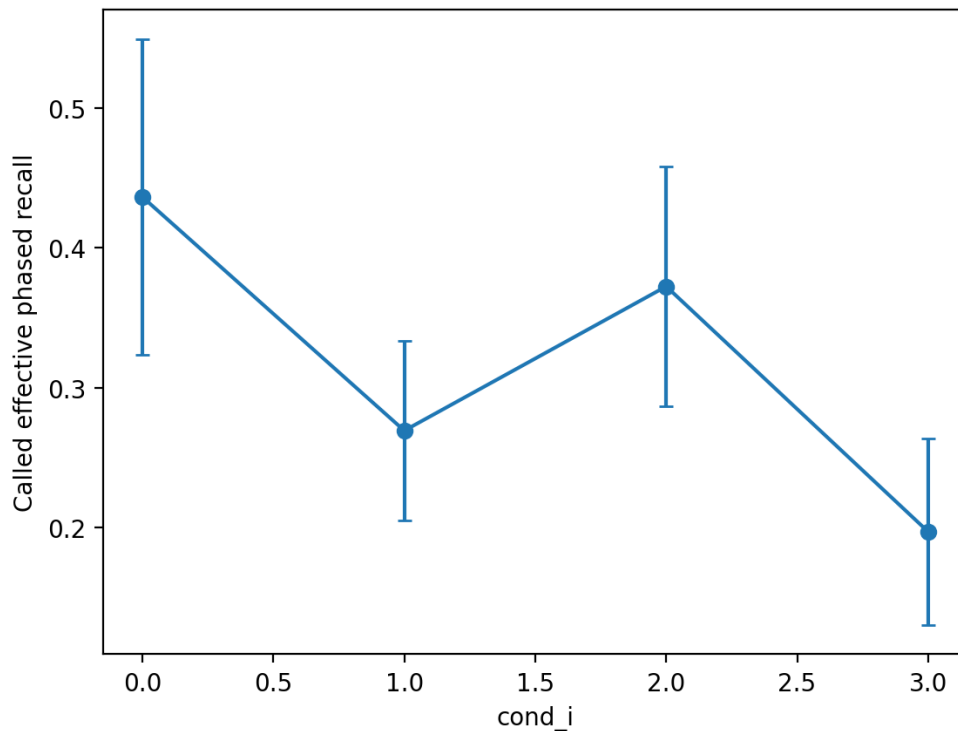


Figure 10.8.2 — Called effective phased recall across duplication × dropout conditions. The combined duplication + dropout condition produces the lowest end-to-end phased recall, showing that the two stressors interact to create a substantially more difficult phasing problem than either one alone.

Figure 10.8.3 — Called number of phase sets across duplication × dropout conditions.

Phase fragmentation is driven primarily by dropout, but remains high in the combined condition. This indicates that low-coverage gaps are the main cause of broken block continuity, while duplication compounds the end-to-end difficulty in other ways.

Figure 10.8.4 — Called switch error across duplication × dropout conditions.

Switch error should be interpreted cautiously in the interaction study. Although duplication alone produces the highest switch error, the combined condition is much more fragmented and leaves fewer phased adjacencies to compare, making completeness and fragmentation metrics more informative than switch error alone.

Results summary (means over seeds)

- `call_recall`:

- dup=0, dropout=0.0: 0.663
- dup=0, dropout=0.1: 0.605
- dup=5, dropout=0.0: 0.666
- dup=5, dropout=0.1: 0.582
- oracle_effective_phased_recall:
 - 0.925 → 0.857 → 0.918 → 0.847
- oracle_num_phase_sets:
 - 1.8 → 3.2 → 1.8 → 4.0
- called_effective_phased_recall:
 - 0.436 → 0.269 → 0.373 → 0.197
- called_shared_het_recall:
 - 0.590 → 0.529 → 0.593 → 0.501
- called_phasing_rate_shared_het:
 - 0.819 → 0.514 → 0.778 → 0.410
- called_phase_accuracy:
 - 0.890 → 0.981 → 0.802 → 0.940
- called_switch_error:
 - 0.122 → 0.006 → 0.207 → 0.062
- called_num_phase_sets:
 - 3.4 → 6.2 → 2.6 → 6.2

Taken together, these results show that the combined duplication + dropout condition is substantially worse than either single stressor alone, and that the strongest compounded loss occurs in the called regime rather than in the oracle regime.

Key observations

- O1 (The combined condition is the worst end-to-end setting): Called effective phased recall decreases from 0.436 in the baseline to 0.269 with dropout alone, 0.373 with duplication alone, and 0.197 when both stressors are combined. This shows that duplication and dropout together create a materially more difficult end-to-end phasing condition than either stressor in isolation.
- O2 (Dropout remains the main driver of phasing fragmentation): In the oracle regime, duplication alone has little effect (oracle_num_phase_sets = 1.8), whereas dropout increases fragmentation (1.8 → 3.2) and the combined condition increases it further (4.0). This indicates that coverage gaps are the primary cause of broken phase-block connectivity.

- O3 (The interaction is strongest in the called regime): Relative to dropout alone, the combined condition further reduces `call_recall` (0.605 → 0.582), `called_shared_het_recall` (0.529 → 0.501), and `called_phasing_rate_shared_het` (0.514 → 0.410). This shows that duplication compounds dropout by worsening both variant recovery and phasing completeness on surviving shared heterozygous sites.
- O4 (The compounded weakness is mixed rather than purely phaser-limited): Oracle effective phased recall decreases only modestly from 0.857 to 0.847 when duplication is added on top of dropout, but called effective phased recall decreases more substantially from 0.269 to 0.197. Therefore, the main interaction effect is not purely within phasing itself; it arises more strongly from the end-to-end combination of reduced callable overlap and weaker phasing connectivity.
- O5 (Switch error is not the main interaction signal): Although duplication alone produces the highest called switch error, the combined condition does not. This should not be interpreted as an improvement, because the combined condition is much more fragmented and leaves fewer adjacent phased heterozygous pairs to compare. In this interaction study, completeness and fragmentation metrics are more informative than switch error alone.

Optimization implication

The duplication × dropout interaction suggests that the most important optimization opportunities under compounded stress are not limited to WhatsHap read selection. Preserving phasing evidence remains important (e.g., avoiding overly strict `min_baseq` filtering), but the interaction also indicates that upstream variant recovery in ambiguous and low-coverage regions is a major limiting factor. This motivates future tuning of calling-side thresholds and, potentially, more region-aware handling of ambiguous reads near coverage gaps.

Takeaway

Duplicated regions and coverage dropout interact to produce a more severe end-to-end phasing weakness than either stressor alone. Dropout drives fragmentation, while duplication further reduces callable overlap and phasing completeness in the called regime. This makes the interaction a useful diagnostic for optimization: it shows that practical improvements will likely require both careful preservation of phasing evidence and better upstream handling of difficult regions, rather than relying on a single WhatsHap parameter alone.

10.9 Optimization sweeps (WhatsHap / pipeline parameters)

Aim

Identify parameter settings that improve phasing performance under a composite hard scenario, and quantify trade-offs between phased recall, switch error, fragmentation, and robustness.

Hard scenario

Optimization sweeps were conducted on a fixed composite stress condition designed to combine several realism knobs: - duplicated regions (`dup_segments = 5`) - coverage dropout (`dropout_fraction = 0.1`, `dropout_block_len = 1000`) - correlated error bursts (`burst_prob = 0.6`, `burst_count = 2`, `burst_len = 300`, `burst_mult = 8.0`) - truth indels (`num_indels = 120`, SNP-only phasing/evaluation enabled) - `ref_length = 120 kb` - `num_snps = 1200` - `num_reads = 300`

This hard scenario was chosen to test whether WhatsHap-side tuning can recover performance under combined realistic stresses rather than under isolated perturbations.

10.9.1 Sweep A: `max_coverage`

Setup

- Varied: `max_coverage` $\in \{10, 15, 25, 40\}$
- Fixed: all other hard-scenario parameters

Figures to include

- **Figure 10.9.1.1 — Called effective phased recall vs `max_coverage`.**
- **Figure 10.9.1.2 — Called switch error vs `max_coverage`.**
- **Figure 10.9.1.3 — Called number of phase sets vs `max_coverage`.**
- **Figure 10.9.1.4 — Total pipeline runtime vs `max_coverage`.**

Figure 10.9.1.1 — Called effective phased recall vs `max_coverage`.

Called effective phased recall remains essentially flat across the `max_coverage` sweep, indicating that increasing the WhatsHap read-selection cap does not provide a meaningful gain under the current hard scenario.

Figure 10.9.1.2 — Called switch error vs `max_coverage`.

Called switch error shows no meaningful dependence on `max_coverage`, further indicating that read-selection depth is not the limiting factor for phasing correctness in the current hard scenario.

Figure 10.9.1.3 — Called number of phase sets vs `max_coverage`.

Phase-set fragmentation remains unchanged across the `max_coverage` sweep, showing that retaining more reads does not improve haplotype continuity in this stress condition.

Figure 10.9.1.4 — Total pipeline runtime vs `max_coverage`.

Runtime increases substantially as `max_coverage` increases, but phasing performance remains nearly unchanged. This indicates that higher `max_coverage` mainly adds compute cost rather than practical benefit.

Results summary

- `call_recall`: 0.5895 at all `max_coverage` settings
- `called_effective_phased_recall`: 0.2917 → 0.2919 → 0.2919 → 0.2919
- `called_switch_error`: 0.0056 at all `max_coverage` settings
- `called_num_phase_sets`: 9.6 at all `max_coverage` settings
- `called_shared_het_recall`: 0.5098 at all `max_coverage` settings
- `called_phasing_rate_shared_het`: 0.5860 → 0.5864 → 0.5864 → 0.5864
- `called_phase_accuracy`: 0.9748 at all `max_coverage` settings
- `oracle_effective_phased_recall`: 0.8256 → 0.8298 → 0.8300 → 0.8300
- `oracle_num_phase_sets`: 6.4 → 6.2 → 6.2 → 6.2
- `time_total_sec`: 3.76 → 4.37 → 6.30 → 6.39

Key observations

- O1 (Changing `max_coverage` has almost no effect on phasing performance): Called effective phased recall, switch error, phase-set count, and phasing-rate-on-shared-hets remain essentially unchanged across the full `max_coverage` sweep.
- O2 (Runtime increases without measurable accuracy gain): Total runtime increases substantially as `max_coverage` increases, but this additional compute does not translate into improved end-to-end phasing metrics.
- O3 (The read-selection cap is not the active bottleneck in this hard scenario): Although higher `max_coverage` retains more reads internally, the resulting phasing output is almost identical. This indicates that the limiting factor is not insufficient retained read coverage at the phasing stage.
- O4 (Optimization headroom from `max_coverage` is negligible): Under the current hard scenario, increasing the WhatsHap read-selection cap does not provide a practically useful optimization lever.

Takeaway Adjusting `max_coverage` does not provide a meaningful optimization benefit under the composite hard scenario. Since phasing metrics remain flat while runtime increases, the default or lower `max_coverage` setting is preferable on efficiency grounds.

10.9.2 Sweep B: phasing quality thresholds (`min_mapq` × `min_baseq`)

Setup

- Varied:
 - `min_mapq` ∈ {0, 10, 20}
 - `min_baseq` ∈ {0, 10, 20}
- Fixed: all other hard-scenario parameters

Figures to include

- **Figure 10.9.2.1** — Called effective phased recall across the `min_mapq × min_baseq` grid.
- **Figure 10.9.2.2** — Called switch error across the `min_mapq × min_baseq` grid.
- **Figure 10.9.2.3** — Called number of phase sets across the phasing quality-threshold grid (heatmap).
- **Figure 10.9.2.4** — Oracle effective phased recall across the phasing quality-threshold grid (heatmap).

Figure 10.9.2.1 — Called effective phased recall across the `min_mapq × min_baseq` grid.

Called effective phased recall is consistently higher when `min_baseq = 0` or `10` than when `min_baseq = 20`, while `min_mapq` has little visible effect. This shows that overly strict base-quality filtering reduces usable phasing evidence.

Figure 10.9.2.2 — Called switch error across the `min_mapq × min_baseq` grid.

Called switch error is lowest when `min_baseq = 0` or `10` and higher when `min_baseq = 20`, indicating that stricter base-quality filtering does not improve phasing correctness in this hard scenario and instead weakens phasing evidence.

Figure 10.9.2.3 — Called number of phase sets across the `min_mapq × min_baseq` grid.

Phase fragmentation is substantially lower when `min_baseq = 0` or `10` than when `min_baseq = 20`, showing that strict base-quality filtering breaks block continuity by removing informative allele observations.

Figure 10.9.2.4 — Oracle effective phased recall across the `min_mapq × min_baseq` grid.

Oracle phasing performance is markedly worse when `min_baseq = 20`, confirming that the main threshold effect arises within the phasing stage itself rather than through changes in the upstream callset.

Results summary

- For `min_baseq = 0` or `10`:
 - `called_effective_phased_recall`: 0.2964
 - `called_switch_error`: 0.0007
 - `called_num_phase_sets`: 6.6
 - `called_phasing_rate_shared_het`: 0.6027
 - `oracle_effective_phased_recall`: 0.9402–0.9415
 - `oracle_num_phase_sets`: 3.8
- For `min_baseq = 20`:
 - `called_effective_phased_recall`: 0.2919–0.2923
 - `called_switch_error`: 0.0056
 - `called_num_phase_sets`: 9.6
 - `called_phasing_rate_shared_het`: 0.5864–0.5872
 - `oracle_effective_phased_recall`: 0.8298–0.8319
 - `oracle_num_phase_sets`: 6.2–6.4

- `min_mapq` has negligible effect across the grid.
- `call_recall` remains constant at 0.5895 because variant-calling thresholds are fixed and only phasing-side filters are varied here.

Key observations

- O1 (Base-quality filtering is the main useful phasing-side knob): The grid separates into two clear regimes. Using `min_baseq = 0` or `10` improves called effective phased recall, lowers switch error, and reduces phase fragmentation relative to `min_baseq = 20`.
- O2 (Overly strict `min_baseq` harms both oracle and called phasing): Raising `min_baseq` to 20 reduces oracle effective phased recall from about 0.94 to about 0.83, and increases oracle phase-set count from 3.8 to 6.2–6.4. This shows that strict base-quality filtering discards too much informative phasing evidence.
- O3 (`min_mapq` contributes little under the current hard scenario): Changing `min_mapq` from 0 to 20 produces almost no measurable difference in called or oracle metrics, indicating that mapping-quality filtering is not a strong optimization lever here.
- O4 (The gain from threshold tuning is modest but real): Lowering `min_baseq` improves phasing completeness and continuity, but the absolute gain in called effective phased recall is still small because the dominant loss term remains missing shared heterozygous sites.

Takeaway Phasing quality thresholds provide a limited but real optimization lever under the composite hard scenario. The most useful change is to avoid overly strict base-quality filtering: `min_baseq = 10` (or 0) clearly outperforms `min_baseq = 20`, while `min_mapq` has little practical impact.

10.9.3 Sweep C: calling thresholds (`call_min_mapq` × `call_min_baseq`)

Setup

- Varied:
 - `call_min_mapq` ∈ {0, 10, 20}
 - `call_min_baseq` ∈ {5, 10, 15, 20}
- Fixed:
 - hard-scenario stressors unchanged
 - phasing parameters fixed to the best practical settings identified earlier:
 - * `max_coverage` = 10
 - * `min_mapq` = 20
 - * `min_baseq` = 10

Figures to include

- **Figure 10.9.3.1 — Variant calling recall across the caller-threshold grid (heatmap).**
- **Figure 10.9.3.2 — Called effective phased recall across the `call_min_mapq` × `call_min_baseq` grid.**
- **Figure 10.9.3.3 — Called shared heterozygous recall across the `call_min_mapq` × `call_min_baseq` grid.**
- **Figure 10.9.3.4 — Call precision across the `call_min_mapq` × `call_min_baseq` grid.**

Figure 10.9.3.1 — Calling recall across the `call_min_mapq` × `call_min_baseq` grid.

Calling recall depends strongly on `call_min_baseq` but only weakly on `call_min_mapq`. Moderate base-quality thresholds retain more true SNP evidence, whereas `call_min_baseq` = 20 is overly strict and sharply reduces variant recovery.

Figure 10.9.3.2 — Called effective phased recall across the `call_min_mapq` × `call_min_baseq` grid.

End-to-end phased recall is highest when `call_min_baseq` = 5 or 10, showing that caller-side base-quality filtering has a direct downstream impact on usable phasing performance.

Figure 10.9.3.3 — Called shared heterozygous recall across the `call_min_mapq` × `call_min_baseq` grid.

Called shared heterozygous recall follows the same pattern as calling recall, indicating that the main benefit of moderate caller thresholds is better recovery of true heterozygous SNPs that can subsequently be phased.

Figure 10.9.3.4 — Call precision across the `call_min_mapq` × `call_min_baseq` grid.

Call precision remains very high across the full threshold grid, showing that the gains from moderate caller base-quality thresholds are achieved without a substantial precision penalty.

Results summary

- For `call_min_baseq` = 5 or 10:
 - `call_precision`: ~0.9995
 - `call_recall`: ~0.596–0.597
 - `called_shared_het_recall`: ~0.517–0.518
 - `called_effective_phased_recall`: ~0.3155–0.3157
 - `called_phasing_rate_shared_het`: ~0.609–0.610
 - `called_switch_error`: 0.0000
 - `called_num_phase_sets`: 6.6
- For `call_min_baseq` = 15:
 - `call_recall`: ~0.589–0.590
 - `called_shared_het_recall`: ~0.509–0.510
 - `called_effective_phased_recall`: ~0.307–0.3074
- For `call_min_baseq` = 20:

- `call_precision`: ~0.9991
 - `call_recall`: ~0.3645–0.3647
 - `called_shared_het_recall`: ~0.287
 - `called_effective_phased_recall`: ~0.1144–0.1146
 - `called_phasing_rate_shared_het`: ~0.399
 - `called_num_phase_sets`: 7.0
- `call_min_mapq` has negligible effect across the grid.

Key observations

- O1 (Caller base-quality filtering is a strong optimization lever): Lowering `call_min_baseq` from 20 to 10 or 5 substantially improves call recall, called shared-heterozygous recall, and called effective phased recall. This shows that overly strict caller base-quality filtering removes too much useful variant evidence under the composite hard scenario.
- O2 (Caller mapping-quality filtering has little practical effect): Changing `call_min_mapq` from 0 to 20 produces almost no measurable difference in calling or phasing metrics. Under the current hard scenario, base-quality filtering is therefore the dominant caller-side threshold knob.
- O3 (The gain arises through overlap recovery rather than phasing correctness): Oracle effective phased recall remains fixed (~0.9402), while called phase accuracy and switch error remain near-perfect in the best settings. This indicates that the improvement comes mainly from recovering more true shared heterozygous sites rather than from changing phasing correctness itself.
- O4 (Moderate caller thresholds outperform strict thresholds without sacrificing precision): The best-performing settings (`call_min_baseq` = 5 or 10) retain very high call precision (~0.9995) while increasing end-to-end phased recall. This makes moderate caller base-quality thresholds a practical optimization opportunity in this pipeline.

Takeaway Calling thresholds provide a more effective optimization lever than the WhatsHap-only sweeps tested earlier. In particular, a moderate caller base-quality threshold (`call_min_baseq` = 10, or 5) improves variant recovery and end-to-end phased recall without materially harming precision, while `call_min_baseq` = 20 is clearly too strict under the current hard scenario.

10.9.4 Sweep D: runtime-focused lower `max_coverage`

Setup

- Varied: `max_coverage` ∈ {4, 6, 8, 10, 15}
- Fixed:
 - hard-scenario stressors unchanged

- caller thresholds fixed to a strong practical setting:
 - * `call_min_mapq = 20`
 - * `call_min_baseq = 15`
- phasing thresholds fixed to the best practical settings identified earlier:
 - * `min_mapq = 20`
 - * `min_baseq = 10`

Figures to include

- **Figure 10.9.4.1 — Called effective phased recall vs lower `max_coverage`.**
- **Figure 10.9.4.2 — Called number of phase sets vs lower `max_coverage`.**
- **Figure 10.9.4.3 — Oracle effective phased recall vs lower `max_coverage`.**
- **Figure 10.9.4.4 — Total pipeline runtime vs lower `max_coverage`.**

Figure 10.9.4.1 — Called effective phased recall vs lower `max_coverage`.

Called effective phased recall is essentially unchanged for `max_coverage = 8, 10, 15`, indicating that retained phasing coverage above 8 provides no practical benefit in the current hard scenario.

Figure 10.9.4.2 — Called number of phase sets vs lower `max_coverage`.

Phase fragmentation is stable from `max_coverage = 6` upward, but becomes slightly worse at `max_coverage = 4`, suggesting that very aggressive down-selection can begin to remove useful connectivity.

Figure 10.9.4.3 — Oracle effective phased recall vs lower `max_coverage`.

Oracle phasing performance improves slightly from `max_coverage = 4` to 8, then plateaus. This indicates that a small amount of retained coverage is necessary for full phasing continuity, but higher retained coverage beyond 8 is unnecessary.

Figure 10.9.4.4 — Total pipeline runtime vs lower `max_coverage`.

Runtime is similar for `max_coverage = 6–10` but increases noticeably at 15, showing that a lower retained-coverage cap can reduce compute cost without sacrificing phasing performance.

Results summary

- `call_recall`: 0.5895 at all `max_coverage` settings
- `call_precision`: 0.9994 at all `max_coverage` settings
- `called_effective_phased_recall`:
 - 0.2958 (`max_coverage = 4`)
 - 0.2960 (6)
 - 0.2964 (8)
 - 0.2964 (10)
 - 0.2964 (15)

- `called_switch_error`:
 - 0.0051 → 0.0038 → 0.0007 → 0.0007 → 0.0007
- `called_num_phase_sets`:
 - 6.8 → 6.6 → 6.6 → 6.6 → 6.6
- `called_phase_accuracy`:
 - 0.9604 → 0.9610 → 0.9625 → 0.9625 → 0.9625
- `oracle_effective_phased_recall`:
 - 0.9228 → 0.9348 → 0.9402 → 0.9402 → 0.9402
- `oracle_num_phase_sets`:
 - 5.0 → 3.8 → 3.8 → 3.8 → 3.8
- `time_total_sec`:
 - 3.6877 → 3.6720 → 3.6769 → 3.6774 → 4.2029

Key observations

- O1 (Performance plateaus by `max_coverage = 8`): Called effective phased recall, switch error, phase-set count, and phase accuracy are essentially identical for `max_coverage = 8, 10, 15`. This shows that retained phasing coverage above 8 does not provide additional practical benefit under the current hard scenario.
- O2 (Very low `max_coverage` begins to hurt modestly): At `max_coverage = 4`, oracle effective phased recall and oracle continuity are slightly worse, and called metrics are also marginally lower. This suggests that reducing retained coverage too aggressively can begin to remove useful bridging evidence.
- O3 (The best efficiency region is around `max_coverage = 8–10`): Runtime remains around 3.67 seconds for `max_coverage = 6–10`, but increases to about 4.20 seconds at 15 without any measurable gain in phasing performance.
- O4 (`max_coverage = 15` is unnecessarily expensive): Compared with `max_coverage = 8–10`, the setting 15 increases runtime by roughly 14% while leaving the main phasing metrics unchanged. This makes higher retained coverage unattractive on efficiency grounds.

Takeaway A lower `max_coverage` setting provides a practical runtime optimization under the composite hard scenario. The best trade-off is around `max_coverage = 8` or `10`, where performance is indistinguishable from 15 but runtime is lower. In contrast, `max_coverage = 4` is slightly too aggressive and begins to degrade continuity-related metrics.

10.9.5 Sweep E: fine phasing `min_baseq`

Setup

- Varied: `min_baseq` $\in \{0, 5, 10, 15, 20\}$
- Fixed:
 - hard-scenario stressors unchanged
 - caller thresholds fixed to a strong practical setting:
 - * `call_min_mapq` = 20
 - * `call_min_baseq` = 15
 - phasing parameters otherwise fixed:
 - * `max_coverage` = 10
 - * `min_mapq` = 20

Figures to include

- **Figure 10.9.5.1 — Called effective phased recall vs fine phasing `min_baseq`.**
- **Figure 10.9.5.2 — Called number of phase sets vs fine phasing `min_baseq`.**
- **Figure 10.9.5.3 — Oracle effective phased recall vs fine phasing `min_baseq`.**
- **Figure 10.9.5.4 — Called switch error vs fine phasing `min_baseq`.**

Figure 10.9.5.1 — Called effective phased recall vs fine phasing `min_baseq`.

Called effective phased recall remains flat from `min_baseq` = 0 to 15, then decreases at 20, indicating that only overly strict phasing base-quality filtering harms usable end-to-end phasing output.

Figure 10.9.5.2 — Called number of phase sets vs fine phasing `min_baseq`.

Phase fragmentation is unchanged across `min_baseq` = 0–15 but increases substantially at 20, showing that strict phasing base-quality filtering breaks block continuity by discarding informative allele observations.

Figure 10.9.5.3 — Oracle effective phased recall vs fine phasing `min_baseq`.

Oracle phasing performance is stable up to `min_baseq` = 15 and then drops sharply at 20, confirming that the harmful threshold effect arises within the phasing stage itself rather than through upstream callset changes.

Figure 10.9.5.4 — Called switch error vs fine phasing `min_baseq`.

Called switch error remains very low and stable from `min_baseq` = 0 to 15, but increases at 20, indicating that overly strict phasing base-quality filtering weakens the consistency of retained phasing evidence.

Results summary

- `call_recall`: 0.5895 at all `min_baseq` settings
- `call_precision`: 0.9994 at all `min_baseq` settings

- `called_effective_phased_recall`:
 - 0.2964 (`min_baseq` = 0)
 - 0.2964 (5)
 - 0.2964 (10)
 - 0.2964 (15)
 - 0.2917 (20)
- `called_switch_error`:
 - 0.0007 → 0.0007 → 0.0007 → 0.0007 → 0.0056
- `called_num_phase_sets`:
 - 6.6 → 6.6 → 6.6 → 6.6 → 9.6
- `called_phase_accuracy`:
 - 0.9625 → 0.9625 → 0.9625 → 0.9625 → 0.9748
- `oracle_effective_phased_recall`:
 - 0.9402 → 0.9402 → 0.9402 → 0.9375 → 0.8256
- `oracle_num_phase_sets`:
 - 3.8 → 3.8 → 3.8 → 3.8 → 6.4
- `time_total_sec`:
 - 3.6758 → 3.6505 → 3.6501 → 3.6172 → 3.6158

Key observations

- O1 (Phasing performance is flat from `min_baseq` = 0 to 15): Called effective phased recall, switch error, phase-set count, and phase accuracy are effectively identical across `min_baseq` ∈ {0, 5, 10, 15}. This indicates that moderate variation in phasing base-quality filtering has no measurable effect within this range.
- O2 (The harmful threshold is `min_baseq` = 20): Raising `min_baseq` to 20 reduces called effective phased recall, increases switch error, and increases phase fragmentation. Oracle effective phased recall also drops sharply (0.9402 → 0.8256), confirming that the loss is caused within the phasing stage itself.
- O3 (There is no practical benefit to using values below 10): Since 0, 5, and 10 perform identically, more permissive phasing base-quality thresholds do not yield additional measurable gains under the current hard scenario.
- O4 (`min_baseq` = 10 is the best practical recommendation): Because 10 lies on the same performance plateau as 0, 5, and 15, but is easier to justify as a moderate threshold, it provides the cleanest practical phasing setting for the remainder of the study.

Takeaway The fine phasing `min_baseq` sweep confirms that performance is stable across `min_baseq = 0–15` and degrades only when the threshold becomes too strict (20). A moderate phasing base-quality threshold such as `min_baseq = 10` is therefore the most practical recommendation: it retains full performance while avoiding unnecessary permissiveness and clearly outperforming the overly strict setting.

10.9.6 Default vs optimized comparison

Setup A final confirmation run compared the default hard-scenario configuration against the best practical tuned configuration identified from the preceding optimization sweeps.

- **Default configuration**

- `call_min_mapq = 20`
- `call_min_baseq = 15`
- `max_coverage = 15`
- `min_mapq = 20`
- `min_baseq = 20`

- **Optimized configuration**

- `call_min_mapq = 20`
- `call_min_baseq = 10`
- `max_coverage = 8`
- `min_mapq = 20`
- `min_baseq = 10`

All other hard-scenario stressors were kept fixed.

Figures to include

- **Figure 10.9.6.1 — Called effective phased recall: default vs optimized.**
- **Figure 10.9.6.2 — Called shared heterozygous recall: default vs optimized.**
- **Figure 10.9.6.3 — Called number of phase sets: default vs optimized.**
- **Figure 10.9.6.4 — Total pipeline runtime: default vs optimized.**

Figure 10.9.6.1 — Called effective phased recall: default vs optimized.

The optimized configuration achieves higher end-to-end phased recall than the default hard-scenario setting, confirming that coordinated parameter tuning yields a measurable improvement under compounded realistic stress.

Figure 10.9.6.2 — Called shared heterozygous recall: default vs optimized.

The optimized configuration increases called shared heterozygous recall, indicating that part of the improvement comes from recovering more true heterozygous sites that can subsequently be phased.

Figure 10.9.6.3 — Called number of phase sets: default vs optimized.

The optimized configuration substantially reduces the number of phase sets, showing that it preserves stronger phase-block continuity than the default setting under the hard scenario.

Figure 10.9.6.4 — Total pipeline runtime: default vs optimized.

The optimized configuration is slightly faster than the default while also improving phasing performance, showing that the observed gains are not achieved at the cost of higher compute.

Results summary Means over 5 seeds:

- `time_total_sec`: 4.0614 → 3.9783
- `call_precision`: 0.9994 → 0.9995
- `call_recall`: 0.5895 → 0.5970
- `oracle_effective_phased_recall`: 0.8298 → 0.9402
- `oracle_switch_error`: 0.0083 → 0.0031
- `oracle_num_phase_sets`: 6.2 → 3.8
- `called_effective_phased_recall`: 0.2919 → 0.3041
- `called_shared_het_recall`: 0.5098 → 0.5181
- `called_phasing_rate_shared_het`: 0.5864 → 0.6090
- `called_phase_accuracy`: 0.9748 → 0.9622
- `called_switch_error`: 0.0056 → 0.0007
- `called_num_phase_sets`: 9.6 → 6.6

Key observations

- O1 (The optimized configuration improves end-to-end phased recall): Called effective phased recall increases from 0.2919 to 0.3041, confirming that the tuned configuration provides a real, if moderate, end-to-end gain under the composite hard scenario.
- O2 (The improvement comes mainly from better overlap and phasing completeness): The optimized configuration increases `call_recall` (0.5895 → 0.5970), `called_shared_het_recall` (0.5098 → 0.5181), and `called_phasing_rate_shared_het` (0.5864 → 0.6090). This shows that the gain arises primarily from recovering and retaining more useful phasing evidence rather than from a large change in phase accuracy itself.
- O3 (Fragmentation is substantially reduced): Called phase-set count decreases from 9.6 to 6.6, while oracle phase-set count decreases from 6.2 to 3.8. This indicates that the optimized configuration preserves much stronger block continuity under the hard scenario.
- O4 (The optimized configuration is also slightly faster): Total runtime decreases from 4.0614 s to 3.9783 s, showing that the optimized setting improves performance without increasing compute cost.

Takeaway The final confirmation run shows that the optimized configuration is practically preferable to the default hard-scenario setting. It improves end-to-end phased recall, increases shared-site overlap and phasing completeness, reduces fragmentation, lowers switch error, and slightly reduces runtime. Although the absolute gain in called effective phased recall is moderate, this experiment confirms that coordinated tuning of caller-side and phasing-side parameters can produce a measurable improvement under compounded realistic stress.

10.9.7 Sweep F: local joint search around recommended thresholds

Setup A focused local search was performed around the current best practical threshold settings to test whether the recommended caller/phasing base-quality pair is locally optimal or whether a nearby combination performs better.

- Varied:
 - `call_min_baseq` ∈ {5, 10, 15}
 - `min_baseq` ∈ {5, 10, 15}
- Fixed:
 - `call_min_mapq` = 20
 - `min_mapq` = 20
 - `max_coverage` = 8
 - hard-scenario stressors unchanged

Figures to include

- **Figure 10.9.7.1 — Called effective phased recall across the local joint base-quality grid (heatmap).**
- **Figure 10.9.7.2 — Called shared heterozygous recall across the local caller/phasing base-quality grid.**
- **Figure 10.9.7.3 — Calling recall across the local caller/phasing base-quality grid.**
- **Figure 10.9.7.4 — Total pipeline runtime across the local caller/phasing base-quality grid.**

Figure 10.9.7.1 — Called effective phased recall across the local caller/phasing base-quality grid.

Called effective phased recall is highest whenever `call_min_baseq` = 5 or 10, while varying phasing `min_baseq` between 5 and 15 has no measurable effect. This shows that local sensitivity is driven mainly by the caller-side threshold.

Figure 10.9.7.2 — Called shared heterozygous recall across the local caller/phasing base-quality grid.

Called shared heterozygous recall follows the same pattern as called effective phased recall, indicating that the local optimization gain arises from recovering more true heterozygous sites rather than from changing phasing correctness.

Figure 10.9.7.3 — Calling recall across the local caller/phasing base-quality grid.

Calling recall improves when `call_min_baseq` is reduced from 15 to 10 or 5, while phasing `min_baseq` has no effect as expected. This confirms that the decisive local optimization lever is the caller base-quality threshold.

Figure 10.9.7.4 — Total pipeline runtime across the local caller/phasing base-quality grid.

Runtime varies only slightly across the local search grid, indicating that the local threshold refinement primarily affects accuracy-related metrics rather than compute cost.

Results summary Grouped means over 5 seeds:

- For `call_min_baseq = 5` or `10`:
 - `call_recall`: 0.5970
 - `called_shared_het_recall`: 0.5181
 - `called_effective_phased_recall`: 0.3041
 - `called_phasing_rate_shared_het`: 0.6090
 - `called_switch_error`: 0.0007
 - `called_num_phase_sets`: 6.6
- For `call_min_baseq = 15`:
 - `call_recall`: 0.5895
 - `called_shared_het_recall`: 0.5098
 - `called_effective_phased_recall`: 0.2964
 - `called_phasing_rate_shared_het`: 0.6027
 - `called_switch_error`: 0.0007
 - `called_num_phase_sets`: 6.6
- Across `min_baseq ∈ {5, 10, 15}`, the main called metrics are effectively unchanged for a fixed `call_min_baseq`.

Key observations

- O1 (The recommended setting is locally robust): No combination in the 3×3 local grid outperforms the existing recommended region. The best results are obtained whenever `call_min_baseq = 5` or `10`, regardless of whether phasing `min_baseq` is 5, 10, or 15.
- O2 (Caller base-quality remains the decisive local knob): Lowering `call_min_baseq` from 15 to 10 or 5 improves `call_recall`, `called_shared_het_recall`, and `called_effective_phased_recall`. This confirms that the main optimization gain still comes from recovering more useful variant evidence upstream.
- O3 (Phasing base-quality is insensitive within the moderate range): For fixed caller thresholds, varying phasing `min_baseq` between 5 and 15 has no measurable effect on the main called metrics. This indicates that, once overly strict filtering has been avoided, phasing base-quality is not a sensitive local optimization lever.
- O4 (`call_min_baseq = 10`, `min_baseq = 10` remains the best practical recommendation): Although `call_min_baseq = 5` performs equivalently to 10, the value 10 is easier to justify as a moderate threshold.

Likewise, phasing `min_baseq = 10` lies on the same performance plateau as 5 and 15, making it the cleanest practical choice.

Takeaway The local search confirms that the recommended optimized threshold pair is robust. The main local sensitivity lies in the caller base-quality threshold, where 10 (or 5) is clearly better than 15, while phasing base-quality is effectively flat within the moderate range 5–15. This strengthens confidence that the chosen optimized configuration is not a fragile one-off result.

10.9.8 Sweep G: accuracy/runtime frontier comparison

Setup Representative configurations were compared under the same hard scenario to summarize the practical trade-off between end-to-end phased recall, block continuity, and runtime.

Configurations compared: - **default** - `call_min_mapq = 20` - `call_min_baseq = 15` - `max_coverage = 15`
- `min_mapq = 20` - `min_baseq = 20` - **caller_only** - `call_min_mapq = 20` - `call_min_baseq = 10`
- `max_coverage = 15` - `min_mapq = 20` - `min_baseq = 20` - **phasing_only** - `call_min_mapq = 20` -
`call_min_baseq = 15` - `max_coverage = 8` - `min_mapq = 20` - `min_baseq = 10` - **balanced** - `call_min_mapq = 20`
- `call_min_baseq = 10` - `max_coverage = 8` - `min_mapq = 20` - `min_baseq = 10` - **runtime** - `call_min_mapq = 20`
- `call_min_baseq = 10` - `max_coverage = 6` - `min_mapq = 20` - `min_baseq = 10`

All other hard-scenario stressors were kept fixed.

Figures to include

- **Figure 10.9.8.1** — Called effective phased recall across representative configurations.
- **Figure 10.9.8.2** — Called shared heterozygous recall across representative configurations.
- **Figure 10.9.8.3** — Called number of phase sets across representative configurations.
- **Figure 10.9.8.4** — Total pipeline runtime across representative configurations.

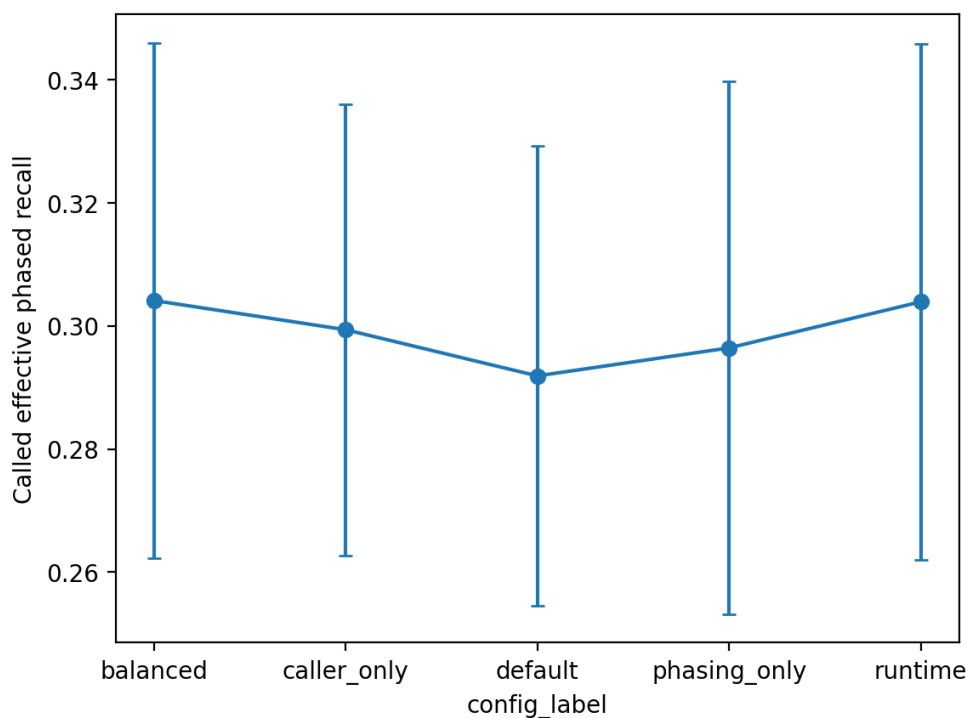


Figure 10.9.8.1 — Called effective phased recall across representative configurations. The balanced configuration achieves the highest end-to-end phased recall, while the runtime-biased configuration performs almost identically. This shows that combining caller-side and phasing-side tuning yields the strongest practical performance.

Figure 10.9.8.2 — Called shared heterozygous recall across representative configurations. Called shared heterozygous recall is highest for the configurations that use the improved caller threshold, confirming that part of the overall gain comes from recovering more true heterozygous sites before phasing.

Figure 10.9.8.3 — Called number of phase sets across representative configurations.

Phase fragmentation is much lower in the phasing-tuned, balanced, and runtime configurations than in the default or caller-only configurations. This shows that phasing-side tuning is the main driver of improved block continuity.

Figure 10.9.8.4 — Total pipeline runtime across representative configurations.

Runtime differences between the representative configurations are modest, but the balanced and runtime-biased settings both remain competitive while outperforming the default configuration on the main phasing metrics.

Results summary Means over 5 seeds:

- **default**
 - time_total_sec: 3.9338
 - call_precision: 0.9994
 - call_recall: 0.5895
 - oracle_effective_phased_recall: 0.8298

- oracle_num_phase_sets: 6.2
- called_effective_phased_recall: 0.2919
- called_shared_het_recall: 0.5098
- called_phasing_rate_shared_het: 0.5864
- called_switch_error: 0.0056
- called_num_phase_sets: 9.6

- **caller_only**

- time_total_sec: 4.0540
- call_precision: 0.9995
- call_recall: 0.5970
- oracle_effective_phased_recall: 0.8298
- oracle_num_phase_sets: 6.2
- called_effective_phased_recall: 0.2994
- called_shared_het_recall: 0.5181
- called_phasing_rate_shared_het: 0.5934
- called_switch_error: 0.0077
- called_num_phase_sets: 9.0

- **phasing_only**

- time_total_sec: 3.7970
- call_precision: 0.9994
- call_recall: 0.5895
- oracle_effective_phased_recall: 0.9402
- oracle_num_phase_sets: 3.8
- called_effective_phased_recall: 0.2964
- called_shared_het_recall: 0.5098
- called_phasing_rate_shared_het: 0.6027
- called_switch_error: 0.0007
- called_num_phase_sets: 6.6

- **balanced**

- time_total_sec: 3.8682
- call_precision: 0.9995
- call_recall: 0.5970
- oracle_effective_phased_recall: 0.9402
- oracle_num_phase_sets: 3.8
- called_effective_phased_recall: 0.3041
- called_shared_het_recall: 0.5181
- called_phasing_rate_shared_het: 0.6090

- `called_switch_error`: 0.0007
- `called_num_phase_sets`: 6.6
- **runtime**
 - `time_total_sec`: 3.8504
 - `call_precision`: 0.9995
 - `call_recall`: 0.5970
 - `oracle_effective_phased_recall`: 0.9348
 - `oracle_num_phase_sets`: 3.8
 - `called_effective_phased_recall`: 0.3039
 - `called_shared_het_recall`: 0.5181
 - `called_phasing_rate_shared_het`: 0.6090
 - `called_switch_error`: 0.0022
 - `called_num_phase_sets`: 6.6

Key observations

- O1 (The balanced configuration provides the best overall trade-off): Among the representative configurations, `balanced` achieves the highest called effective phased recall (0.3041) while also maintaining low fragmentation (6.6 phase sets), very low switch error (0.0007), and lower runtime than the default configuration.
- O2 (Default is dominated by the tuned configurations): The default configuration is worse than `balanced` and `runtime` on called effective phased recall, called shared heterozygous recall, switch error, and fragmentation, while also not being the fastest option. This indicates that the untuned hard-scenario settings are not practically competitive.
- O3 (Caller-only and phasing-only tuning recover complementary parts of the gain): `caller_only` improves overlap-related metrics (`call_recall`, `called_shared_het_recall`) but leaves fragmentation high, whereas `phasing_only` strongly improves continuity and oracle phasing metrics but does not recover additional called-site overlap. The balanced configuration combines both benefits.
- O4 (The runtime-biased configuration is a viable alternative): `runtime` performs almost identically to `balanced` on the main called metrics (0.3039 vs 0.3041 called effective phased recall) while using a slightly lower `max_coverage`. This makes it a reasonable alternative when efficiency is prioritized, although `balanced` remains the cleaner general recommendation.

Takeaway The frontier comparison shows that the best practical performance comes from combining moderate caller-side and phasing-side tuning rather than optimizing either stage in isolation. The balanced configuration is the preferred general setting because it delivers the highest end-to-end phased recall together with strong continuity and low switch error. A slightly more runtime-biased configuration performs almost identically and may be acceptable when efficiency is prioritized, but the default configuration is clearly dominated.

10.9.9 Optimization robustness across scenarios

Setup To test whether the recommended optimized settings generalize beyond the hard tuning scenario, three representative configurations were compared across four scenarios:

Configurations compared: - **default** - **balanced** - **runtime**

Scenarios compared: - **baseline** - **dropout** - **interaction** - **hard**

The aim was to determine whether the recommended tuning is broadly useful or only beneficial in the hard scenario used for parameter selection.

Figures to include

- **Figure 10.9.9.1** — Called effective phased recall across scenarios and representative configurations (heatmap).
- **Figure 10.9.9.2** — Called shared heterozygous recall across scenarios and representative configurations.
- **Figure 10.9.9.3** — Called number of phase sets across scenarios and representative configurations.
- **Figure 10.9.9.4** — Total pipeline runtime across scenarios and representative configurations.

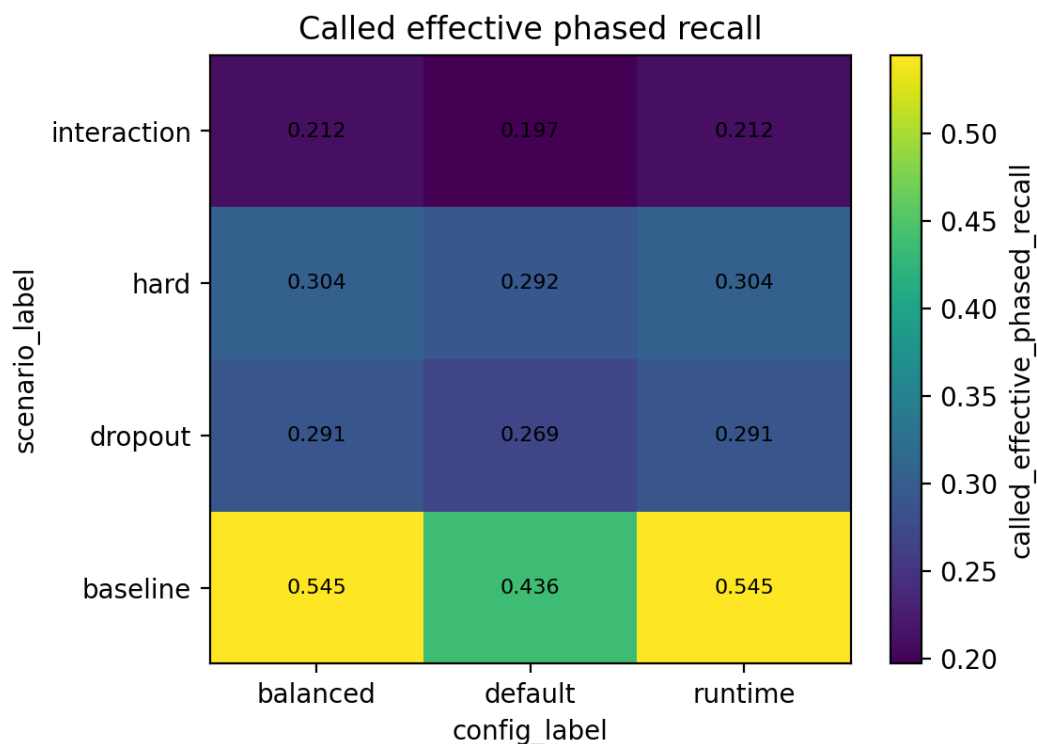


Figure 10.9.9.1 — Called effective phased recall across scenarios and representative configurations. Across all tested scenarios, the balanced and runtime configurations achieve higher end-to-end phased recall than the default configuration, showing that the selected tuning generalizes beyond the hard optimization scenario.

Figure 10.9.9.2 — Called shared heterozygous recall across scenarios and representative configurations.

The tuned configurations consistently improve called shared heterozygous recall, indicating that part of the transferable gain comes from recovering more true heterozygous sites across multiple scenarios.

Figure 10.9.9.3 — Called number of phase sets across scenarios and representative configurations.

The balanced and runtime configurations reduce phase fragmentation relative to the default configuration in all tested scenarios, showing that the tuning also improves block continuity rather than only site recovery.

Figure 10.9.9.4 — Total pipeline runtime across scenarios and representative configurations.

Runtime remains competitive for the tuned configurations across scenarios, and the runtime-biased variant offers nearly identical accuracy with similar or slightly improved efficiency.

Results summary Means over 5 seeds:

- **Baseline**

- default: `called_effective_phased_recall = 0.4364`, `called_shared_het_recall = 0.5902`,
`called_num_phase_sets = 3.4`, `time_total_sec = 2.9850`
- balanced: `0.5448`, `0.6097`, `1.4`, `3.0223`
- runtime: `0.5448`, `0.6097`, `1.4`, `2.9596`

- **Dropout**

- default: `0.2694`, `0.5292`, `6.2`, `2.9768`
- balanced: `0.2908`, `0.5420`, `4.2`, `2.9362`
- runtime: `0.2908`, `0.5420`, `4.2`, `2.9391`

- **Interaction**

- default: `0.1972`, `0.5014`, `6.2`, `3.0043`
- balanced: `0.2118`, `0.5094`, `5.4`, `2.8789`
- runtime: `0.2118`, `0.5094`, `5.4`, `2.9750`

- **Hard**

- default: `0.2919`, `0.5098`, `9.6`, `4.3644`
- balanced: `0.3041`, `0.5181`, `6.6`, `4.2497`
- runtime: `0.3039`, `0.5181`, `6.6`, `4.3050`

Key observations

- O1 (The tuned configurations generalize across scenarios): In all four scenarios, both `balanced` and `runtime` outperform `default` on called effective phased recall and called shared heterozygous recall. This shows that the tuned settings are not overfit to the hard scenario alone.

- O2 (The balanced configuration is the safest general recommendation): `balanced` is never worse than `default` and is either best or tied for best on the main called metrics across all scenarios. This supports its use as the default recommended configuration.
- O3 (The runtime-biased configuration is highly competitive): `runtime` is effectively identical to `balanced` on the main called metrics in this sweep and remains competitive on runtime. This makes it a reasonable alternative when efficiency is prioritized.
- O4 (The gains transfer to both easy and hard cases): The tuned configurations improve performance not only in the hard scenario, but also in the baseline, dropout, and interaction scenarios. This indicates that the optimization is robust across qualitatively different data conditions.

Takeaway The recommended tuned settings are robust across the representative scenarios tested in this study. The balanced configuration is the cleanest general recommendation because it consistently improves end-to-end phased recall and reduces fragmentation relative to the default. A runtime-biased alternative performs almost identically and may be preferred when efficiency is prioritized.

10.9.10 Sweep H: caller `call_min_mapq` rule-out

Setup A final small rule-out sweep was conducted to test whether caller mapping-quality filtering provides meaningful optimization headroom once the other recommended settings are fixed.

- Varied:
 - `call_min_mapq` $\in \{0, 10, 20\}$
- Fixed:
 - `call_min_baseq` = 10
 - `max_coverage` = 8
 - `min_mapq` = 20
 - `min_baseq` = 10
 - hard-scenario stressors unchanged

Figures to include

- **Figure 10.9.10.1** — Called effective phased recall vs `call_min_mapq`.
- **Figure 10.9.10.2** — Calling recall vs `call_min_mapq`.
- **Figure 10.9.10.3** — Called shared heterozygous recall vs `call_min_mapq`.
- **Figure 10.9.10.4** — Call precision vs `call_min_mapq`.

Figure 10.9.10.1 — Called effective phased recall vs `call_min_mapq`.

Called effective phased recall changes only slightly across the caller mapping-quality sweep, indicating that `call_min_mapq` is not a strong optimization lever under the current hard scenario.

Figure 10.9.10.2 — Calling recall vs `call_min_mapq`.

Calling recall remains essentially unchanged across the caller mapping-quality sweep, showing that mapQ filtering has little influence on variant recovery in this setup.

Figure 10.9.10.3 — Called shared heterozygous recall vs `call_min_mapq`.

Called shared heterozygous recall is nearly flat across the caller mapping-quality sweep, confirming that caller mapQ does not materially alter the overlap of useful true heterozygous sites.

Figure 10.9.10.4 — Call precision vs `call_min_mapq`.

Call precision remains extremely high across all caller mapping-quality settings, indicating that stricter mapQ filtering does not provide a meaningful precision advantage in this pipeline.

Results summary Means over 5 seeds:

- `call_min_mapq = 0`
 - `call_precision = 0.9995`
 - `call_recall = 0.5962`
 - `called_shared_het_recall = 0.5171`
 - `called_effective_phased_recall = 0.3155`
 - `called_phasing_rate_shared_het = 0.6102`
 - `called_phase_accuracy = 1.0000`
 - `called_switch_error = 0.0000`
 - `time_total_sec = 4.1980`
- `call_min_mapq = 10`
 - `call_precision = 0.9995`
 - `call_recall = 0.5970`
 - `called_shared_het_recall = 0.5181`
 - `called_effective_phased_recall = 0.3157`
 - `called_phasing_rate_shared_het = 0.6094`
 - `called_phase_accuracy = 1.0000`
 - `called_switch_error = 0.0000`
 - `time_total_sec = 4.1432`
- `call_min_mapq = 20`
 - `call_precision = 0.9995`
 - `call_recall = 0.5970`
 - `called_shared_het_recall = 0.5181`
 - `called_effective_phased_recall = 0.3041`
 - `called_phasing_rate_shared_het = 0.6090`
 - `called_phase_accuracy = 0.9622`

- `called_switch_error = 0.0007`
- `time_total_sec = 4.1246`

Key observations

- O1 (Caller mapping-quality filtering has little effect on overlap recovery): Calling recall and called shared heterozygous recall remain essentially unchanged across `call_min_mapq` $\in \{0, 10, 20\}$. This indicates that caller mapping-quality filtering does not materially alter the amount of useful variant evidence reaching the phasing stage.
- O2 (The optimization signal is much weaker than for caller base-quality): In contrast to the earlier caller base-quality sweep, changing `call_min_mapq` produces only very small differences in the main called metrics. This shows that caller mapping-quality is not a strong optimization lever in the current pipeline.
- O3 (No compelling benefit is obtained from stricter caller mapQ filtering): Although `call_min_mapq = 20` shows a slight reduction in called effective phased recall, the effect is small and not accompanied by meaningful changes in call precision or shared-site recall. This suggests that aggressive caller mapQ filtering provides no practical advantage here.

Takeaway Caller mapping-quality filtering does not provide meaningful optimization headroom under the current hard scenario. Unlike caller base-quality, which showed a clear and useful tuning signal, `call_min_mapq` has only weak effects on the main end-to-end phasing metrics. This supports treating caller mapQ as a low-priority or effectively negligible optimization knob in this pipeline.

10.9.11 DP-scaling under hard conditions

Aim Assess how the downstream phasing problem scales with increasing SNP density under the composite hard scenario, and determine whether the tuned configuration changes the runtime growth of the solve stage.

Setup A small scaling study was conducted under the fixed hard scenario, varying only the number of SNPs:

- `num_snps` $\in \{400, 800, 1200, 1600\}$

Two configurations were compared: - **default** - `call_min_baseq = 15` - `max_coverage = 15` - `min_baseq = 20`
- **balanced** - `call_min_baseq = 10` - `max_coverage = 8` - `min_baseq = 10`

All other hard-scenario stressors were kept fixed.

Figures to include

- **Figure 10.9.11.1 — Called solve time vs SNP count under default and optimized configurations.**

- **Figure 10.9.11.2 — Oracle solve time vs SNP count under default and optimized configurations.**
- **Figure 10.9.11.3 — Called effective phased recall vs SNP count under default and optimized configurations.**
- **Figure 10.9.11.4 — Called number of phase sets vs SNP count under default and optimized configurations.**

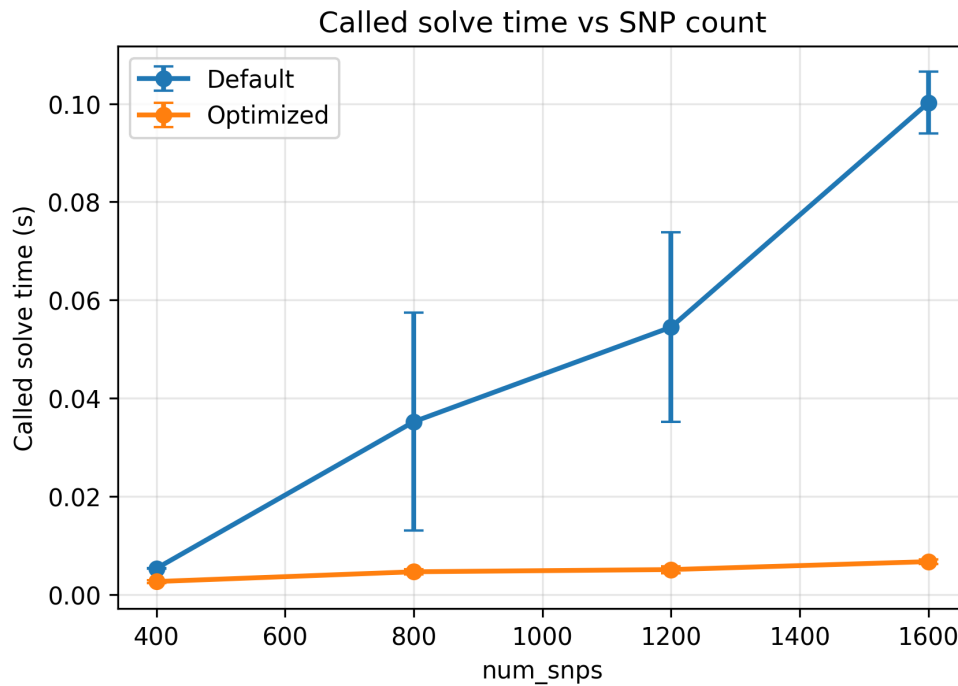


Figure 10.9.11.1 — Called solve time vs SNP count under default and optimized configurations. Called solve time increases with SNP density under both configurations, but grows much more steeply under the default setting. This shows that the tuned configuration makes the residual phasing problem easier to solve as variant density increases.

Figure 10.9.11.2 — Oracle solve time vs SNP count under default and optimized configurations.

Oracle solve time also increases with SNP density, confirming that the solver becomes more expensive as the phasing-only problem grows. The optimized configuration remains substantially faster, indicating that its benefit is not solely due to changes in the called callset.

Figure 10.9.11.3 — Called effective phased recall vs SNP count under default and optimized configurations.

Across the scaling range, the optimized configuration maintains slightly higher end-to-end phased recall than the default configuration, showing that its runtime benefit is not obtained by sacrificing phasing quality.

Figure 10.9.11.4 — Called number of phase sets vs SNP count under default and optimized configurations.

The optimized configuration generally produces fewer phase sets than the default configuration as SNP density increases, indicating better phase-block continuity and a less fragmented residual phasing problem.

Results summary Means over 3 seeds:

- **optimized**

- `called_time_solve_sec`: 0.0026 → 0.0046 → 0.0051 → 0.0067
- `oracle_time_solve_sec`: 0.0044 → 0.0066 → 0.0091 → 0.0114
- `called_effective_phased_recall`: 0.3071 → 0.3705 → 0.3295 → 0.3280
- `called_num_phase_sets`: 6.0 → 5.7 → 7.3 → 6.3

- **Default**

- `called_time_solve_sec`: 0.0053 → 0.0352 → 0.0545 → 0.1002
- `oracle_time_solve_sec`: 0.0114 → 0.0533 → 0.0863 → 0.1407
- `called_effective_phased_recall`: 0.2935 → 0.3539 → 0.3164 → 0.3141
- `called_num_phase_sets`: 8.3 → 10.7 → 10.0 → 7.7

Key observations

- O1 (Solve time increases with SNP density): In both oracle and called regimes, solve time increases as `num_snps` increases from 400 to 1600, showing that the downstream phasing problem becomes more expensive as variant density rises.
- O2 (The optimized configuration scales much better than default): The growth in solve time is much steeper under the default configuration than under the optimized configuration. At `num_snps = 1600`, default solve time is substantially higher than optimized in both oracle and called regimes, indicating that the tuned settings make the residual phasing problem easier to solve.
- O3 (optimized remains better on phasing continuity as the problem grows): Across the scaling range, the optimized configuration generally maintains fewer phase sets and slightly better phased recall than the default configuration. This suggests that the tuning improves not only final phasing output but also the structure of the instance presented to the solver.
- O4 (Solve time still does not dominate total runtime end-to-end): Although solve time grows with SNP density, total phasing runtime remains dominated by preprocessing/readset construction in the current research pipeline. Therefore, the main value of this result is algorithmic interpretation rather than end-to-end runtime engineering.

Takeaway The DP-scaling study shows that increasing SNP density makes the phasing solve stage progressively more expensive, but also that the optimized tuned configuration scales substantially better than the default configuration. This suggests that practical tuning can improve not only phased output quality but also the difficulty of the residual wMEC-style problem faced by the solver.

Overall takeaway

Under the current hard scenario, optimization headroom exists but is uneven across knobs. WhatsHap-side tuning provides only limited gains: increasing `max_coverage` does not help, and the main useful phasing-side adjust-

ment is to avoid overly strict `min_baseq` filtering. In contrast, caller-side tuning is more impactful: lowering `call_min_baseq` from 20 to a moderate value such as 10 improves variant recovery, shared heterozygous overlap, and end-to-end phased recall without materially reducing precision. Taken together, these results indicate that practical optimization in this pipeline is not purely a WhatsHap parameter problem; meaningful gains are more likely to come from a combination of permissive phasing evidence retention and improved upstream variant recovery.

10.10 Summary of results (cross-experiment synthesis)

10.10.1 Attribution summary (oracle vs called)

Taken together, the oracle-vs-called comparison provides a consistent attribution picture across the stress studies.

- **Duplications:** primarily a **called-regime phasing/evidence-quality stressor**, rather than a strong caller-limited bottleneck. Calling recall remains nearly unchanged across duplication levels, and oracle effective phased recall remains high and stable. The main degradation appears only at the strongest duplication level, where called phasing completeness and phase accuracy worsen without a corresponding drop in shared-site recall. This indicates that duplicated regions mainly weaken the consistency of phasing evidence on surviving called sites.
- **Coverage dropout:** primarily a **fragmentation-dominated stressor**, becoming **mixed** at high severity. Oracle phase-set counts increase strongly with dropout fraction and oracle effective phased recall declines, showing that dropout directly breaks read connectivity even when correct sites are supplied. In the called regime, performance degrades more strongly still, and at severe dropout both shared-site recall and phasing completeness collapse. This makes dropout the clearest example of a connectivity bottleneck that later becomes both a calling and phasing bottleneck.
- **Correlated error bursts:** a **weak and somewhat noisy called-regime correctness stressor** under the current parameterization. Oracle metrics remain stable, calling recall is broadly unchanged, and the main visible effect is a dip in called-regime phase accuracy and switch stability at intermediate burst probability. Because the response is non-monotonic, this stressor should be interpreted cautiously and does not currently provide a strong optimization signal.
- **Read length model:** a **mixed continuity/overlap trade-off** rather than a clear stressor or optimization knob. The lognormal model improves calling recall, shared-site recall, and phase-block continuity, but these gains are largely offset by slightly worse called-regime phasing correctness. This shows that better connectivity alone does not guarantee a strong end-to-end gain.
- **Truth indels with SNP-only policy:** primarily a **methodology-validity result**, not a performance stressor. Oracle SNP-phasing metrics remain stable and called SNP-level metrics stay within a comparable range

across indel settings, indicating that `phase_snps_only` and `eval_snps_only` successfully preserve interpretable SNP phasing evaluation even when truth indels are present.

- **Duplication × dropout interaction: a mixed compounded weakness**, strongest in the called regime. Dropout remains the main source of fragmentation, but duplication further worsens both called-site overlap and phasing completeness on surviving sites. This interaction shows that realistic end-to-end failure modes arise most clearly when ambiguous mapping and low-coverage gaps are combined.

Overall, the attribution pattern is consistent: whenever oracle performance remains strong but called performance degrades, the dominant bottleneck lies in the overlap and quality of called heterozygous sites. When oracle performance also degrades, the stressor is directly harming phase-block connectivity or phasing completeness.

10.10.2 Most impactful stressors

Across the single-knob and interaction studies, the realism knobs differ substantially in practical impact.

- **Most impactful overall: coverage dropout.** It produces the clearest and strongest reduction in called effective phased recall, strongly increases phase fragmentation, and eventually also reduces calling recall. It is the most important isolated stressor in this study.
- **Most impactful compounded weakness: duplication × dropout interaction.** While duplication alone is relatively mild, its combination with dropout creates a substantially worse end-to-end condition than either stressor alone, especially in the called regime. This reveals a realistic mixed failure mode involving both reduced callable overlap and weaker phasing connectivity.
- **Moderate impact: read length model and strong duplication.** The read length model changes continuity and overlap in opposite directions, while strong duplication mainly affects called-regime phasing quality at the highest severity.
- **Weak / noisy impact: correlated error bursts.** Under the current parameterization, bursts do not produce a strong monotonic degradation trend and therefore are less informative as a practical optimization target.
- **Methodological rather than stress-inducing: truth indels under SNP-only policy.** Their main value is to validate that the evaluation remains interpretable under indel-containing truth sets.

If stressors are ranked by effect on the main end-to-end metric (`called_effective_phased_recall`), the most important findings are:

1. **dropout**,
2. **duplication × dropout interaction**,
3. **strong duplication / read-length trade-off effects**,
4. **bursts (weak/noisy)**,

with the indel study serving primarily as an evaluation-robustness check rather than a degradation study.

10.10.3 Optimization summary

Optimization sweeps under the composite hard scenario showed that optimization headroom exists, but is uneven across knobs.

A small runtime breakdown of the called phasing stage was also examined for representative configurations under the hard scenario:

Configuration	Build ReadSet (s)	Read Selection (s)	Called Phasing Total	
			Solve (s)	(s)
Optimized	0.3679	0.0061	0.0056	0.3854
Default	0.3734	0.0023	0.0610	0.4433
Speed-oriented	0.3685	0.0050	0.0030	0.3827

This runtime breakdown shows that, in the current research pipeline, the largest share of called phasing runtime is spent in readset construction, while read selection and the downstream solve stage contribute smaller but configuration-dependent fractions. In particular, the default configuration has a noticeably larger solve-stage cost than the tuned configurations, whereas the optimized and speed-oriented settings are dominated even more strongly by readset construction. This result should be interpreted cautiously, because the present study uses a custom adaptor to support oracle/called benchmarking and unified reporting rather than the standard production-facing WhatsHap front-end. Nevertheless, it suggests that evidence extraction can be a significant practical cost in WhatsHap-based phasing workflows, and that the tuned configurations improve runtime mainly through a better overall phasing setup rather than through a large reduction in the core solve stage alone.

The main optimization findings are:

- **Useful caller-side knob:** `call_min_baseq`. Lowering the caller base-quality threshold from 20 to a moderate value such as 10 substantially improves calling recall, shared heterozygous overlap, and end-to-end phased recall without materially harming precision. This is the strongest optimization signal found in the study.
- **Useful phasing-side knob:** `min_baseq`, but mainly as a **rule-out of overly strict filtering**. Phasing performance is stable for `min_baseq` = 0–15, but degrades at 20, indicating that strict phasing base-quality filtering discards too much informative evidence. A moderate setting such as `min_baseq` = 10 is the cleanest practical recommendation.
- **Useful runtime knob:** `max_coverage`. Increasing `max_coverage` beyond 8–10 does not improve phasing performance, while lower values such as 8 preserve full performance with lower runtime. This makes `max_coverage` = 8 a practical efficiency setting.
- **Weak / negligible knobs:** caller `call_min_mapq`, phasing `min_mapq`, and high `max_coverage` values above the plateau. These do not provide meaningful practical gains under the tested hard scenario.

- **Robustness of tuning:** the tuned configurations generalize across baseline, dropout, interaction, and hard scenarios. In all tested cases, the balanced tuned configuration improves called effective phased recall and reduces fragmentation relative to default, while a runtime-biased configuration performs almost identically and remains a viable alternative when efficiency is prioritized.
- **Algorithm-facing scaling result:** increasing SNP density makes the solve stage more expensive, but the balanced tuned configuration scales much better than the default configuration. This suggests that practical tuning not only improves phased output quality, but also reduces the difficulty of the residual wMEC-style problem faced by the solver.

Based on the optimization sweeps, the recommended practical settings for this pipeline are:

- **Recommended `max_coverage`:** 8
- **Recommended phasing thresholds:** `min_mapq = 20`, `min_baseq = 10`
- **Recommended calling thresholds:** `call_min_mapq = 20` (or leave at default, as the effect is negligible), `call_min_baseq = 10`

A slightly more runtime-biased alternative (`max_coverage = 6`, with the same caller/phasing quality thresholds) performs almost identically on the main accuracy metrics and may be acceptable when efficiency is prioritized.

Future work could compare the custom adaptor against the standard WhatsHap front-end directly, or optimize the adaptor separately if pipeline-level runtime becomes a primary objective.

11. Discussion

11.1 Discussion of main findings

This study set out to understand WhatsHap performance under realistic long-read conditions, identify where performance is lost in an end-to-end pipeline, and explore whether practical optimization opportunities exist. The experimental results show that these goals were achieved at three levels: attribution, stress characterization, and optimization.

First, the baseline depth sweep showed a consistent attribution pattern. When correct variant sites are supplied (oracle regime), WhatsHap phasing is already highly accurate, and the main limitation at lower depth is incompleteness due to missing connectivity. In contrast, in the called regime, a large proportion of the end-to-end performance loss is caused by the overlap of correctly called heterozygous variants rather than by the phasing algorithm itself. This establishes an important baseline conclusion: in practical settings, the phaser is not always the dominant bottleneck, and improvements in end-to-end phased recall often depend as much on variant recovery as on phasing quality.

Second, the realism-stressor studies revealed that different perturbations harm the pipeline in qualitatively different ways. Coverage dropout emerged as the clearest and strongest isolated stressor, primarily by fragmenting phase blocks and reducing phasing completeness, and at high severity by also reducing variant recovery. Duplicated regions, in contrast, were relatively mild under the current parameterization, with their main impact appearing only at the highest setting and mostly in the called regime. Correlated error bursts produced a weaker and more variable signal than expected, while the read-length model showed a mixed trade-off: better continuity and overlap, but no substantial end-to-end gain because correctness became noisier. The indel study served mainly as a methodology-validation result, showing that SNP-only safeguards preserve interpretable evaluation when truth indels are present.

Third, the interaction study demonstrated that the most informative failure modes may not come from isolated knobs alone. Duplication combined with dropout produced a substantially worse end-to-end condition than either stressor individually, especially in the called regime. This indicates that realistic phasing difficulty can arise from compounded weaknesses: ambiguous mapping worsens low-coverage conditions by simultaneously reducing callable overlap and weakening phasing completeness. This result was particularly useful because it linked the stress studies to the optimization studies: the key practical weaknesses are mixed, rather than purely phaser-limited.

Finally, the optimization experiments showed that useful but limited optimization headroom exists. The strongest positive signal came from caller-side base-quality tuning: reducing `call_min_baseq` from an overly strict value improved variant recovery, shared heterozygous overlap, and end-to-end phased recall without materially harming precision. On the phasing side, the clearest lesson was not to over-filter: `min_baseq = 20` was consistently harmful, while moderate values such as 10 lay on a stable performance plateau. Retained coverage (`max_coverage`) also showed a practical runtime trade-off, with values around 8–10 preserving performance while avoiding unnecessary compute. In contrast, both caller and phasing mapping-quality thresholds provided little useful headroom. Taken together, these results suggest that practical optimization is not a matter of one magic WhatsHap knob, but rather a combination of preserving useful evidence, recovering more overlapping heterozygous variants upstream,

and avoiding overly conservative filtering.

An additional algorithm-facing result came from the SNP-density scaling experiment. As variant density increased, the downstream solve stage became more expensive, but the balanced tuned configuration scaled substantially better than the default configuration. This suggests that tuning does not only improve final phasing output; it also changes the difficulty of the residual wMEC-style problem presented to the solver. Although total runtime in the research pipeline remained dominated by the project-specific readset-construction adaptor, this scaling result still supports the conclusion that practical tuning can improve both output quality and solver-facing problem structure.

Overall, the findings support a balanced interpretation of WhatsHap in realistic long-read pipelines. WhatsHap itself is robust when correct variant evidence is available, but end-to-end performance depends strongly on how much useful evidence survives calling, filtering, and connectivity disruptions. Therefore, optimization opportunities exist, but they are fundamentally pipeline-level rather than purely phaser-internal.

11.2 Practical implications for WhatsHap-based phasing

From a practical perspective, the results suggest several concrete recommendations for WhatsHap-based long-read phasing pipelines.

The first implication is that end-to-end phasing quality should not be interpreted using phasing metrics alone. In many conditions tested here, especially at moderate depth and under compounded stress, the dominant loss arose before or alongside phasing, through reduced overlap of correctly called heterozygous variants. This means that practitioners should evaluate both variant recovery and phasing quality together. A strong oracle result does not guarantee equally strong end-to-end performance if the called VCF fails to preserve enough informative heterozygous sites.

The second implication is that conservative filtering can be counterproductive. On the calling side, overly strict base-quality filtering removed too much true variant evidence and substantially reduced downstream phased recall. On the phasing side, strict base-quality filtering similarly weakened connectivity and fragmentation metrics once the threshold became too aggressive. In the current pipeline, moderate settings performed better than highly conservative ones. This supports a practical strategy of preserving useful evidence unless there is a strong reason to discard it.

The third implication is that practical tuning should combine caller-side and phasing-side decisions. The best-performing general configuration was not achieved by optimizing the phaser in isolation, nor by optimizing the caller in isolation, but by combining both types of tuning. The frontier and transferability experiments further showed that this balanced tuned configuration generalizes across baseline, dropout, interaction, and hard conditions. Therefore, the study supports recommending a small tuned configuration set rather than a single-parameter adjustment.

A practical configuration supported by the experiments is: - call_min_baseq = 10 - call_min_mapq = 20 (or default, since the effect is negligible) - max_coverage = 8 - min_mapq = 20 - min_baseq = 10

A slightly more runtime-biased alternative with `max_coverage = 6` performed almost identically in the tested scenarios and may be acceptable when efficiency is prioritized.

The fourth implication is that not every intuitive tuning direction is worth pursuing. Increasing `max_coverage` above the plateau did not help. Caller-side and phasing-side mapQ thresholds were also weak knobs. This is important because negative findings are still useful for practical tuning: they narrow the search space and show where optimization effort is unlikely to be rewarded.

Finally, the results suggest that some of the most important remaining opportunities lie upstream of the phasing solver itself. Whenever performance loss was primarily driven by missing shared heterozygous sites, improvements in variant recovery are likely to matter more than further refinement of phasing thresholds. This does not diminish the value of WhatsHap tuning, but it clarifies where future practical gains are most likely to come from.

11.3 Limitations of the present study

Several limitations should be acknowledged when interpreting the results.

First, the study uses a synthetic benchmarking pipeline rather than a real biological truth set. This was a deliberate design choice because it enabled precise control over stressors such as duplication, dropout, and correlated error bursts, and made oracle-vs-called attribution possible. However, synthetic truth still simplifies some aspects of real sequencing data and genome structure. As a result, the absolute metric values reported here should be interpreted as controlled experimental measurements rather than direct predictions of production performance on all real datasets.

Second, some realism knobs produced weaker or noisier signals than expected. In particular, correlated error bursts did not show a strong monotonic degradation trend under the tested parameterization, and duplication alone was milder than expected except at the strongest setting. This does not invalidate the experiments, but it means that some conclusions are better interpreted as ruling out large effects under the current settings rather than as definitive evidence that the stressor is unimportant in general.

Third, the phasing pipeline used a custom adaptor layer to construct the `ReadSet` consumed by the vendored WhatsHap core. This adaptor was necessary for the project's experimental interface, including support for oracle/called benchmarking, unified JSON provenance, and controlled SNP-only evaluation. Runtime profiling showed that the dominant runtime cost in the research pipeline lies in this adaptor rather than in the solver itself. For this reason, the runtime results should be interpreted carefully: they are informative about the project's benchmarking infrastructure, but they do not necessarily represent the standard production-facing WhatsHap front-end directly.

Fourth, although the optimization sweeps were broad enough to identify practical recommendations, they do not exhaust the full parameter space. The recommended settings should therefore be interpreted as strong experimentally supported candidates rather than mathematically proven optima. The local search reduced this uncertainty substantially by showing that the selected threshold pair is locally robust, but broader or more adaptive searches could still be explored in future.

Fifth, the transferability and scaling studies were intentionally kept compact to remain tractable within the project scope. They are sufficient to support the main conclusions about robustness and solver-facing scaling, but they do

not replace a full industrial-scale benchmarking campaign.

11.4 Future work

Several natural extensions follow from this study.

A first direction is to test the balanced tuned configuration on broader and more realistic datasets, including real long-read data or more complex synthetic references. This would help determine how far the present recommendations transfer beyond the controlled pipeline used here.

A second direction is to study upstream variant recovery more directly. Since many end-to-end losses were shown to be caller-limited, a natural extension would be to compare different variant callers, different calling models, or more adaptive caller-side filtering strategies. The present work suggests that this may be a more promising route to further gains than continued refinement of phasing thresholds alone.

A third direction is to deepen the algorithm-facing analysis of the residual phasing problem. The SNP-density scaling study already suggests that tuning can change solver difficulty, but a richer study could investigate how variant density, read connectivity, and overlap structure jointly affect the residual wMEC problem instance presented to the DP solver.

A fourth direction is to compare the project-specific adaptor against the standard WhatsHap front-end more directly. This would help separate properties of the benchmarking interface from properties of the production-facing workflow. Such a comparison is especially relevant for runtime interpretation.

A fifth direction is to investigate region-aware optimization strategies under compounded stress. The duplication $\times$ dropout interaction suggests that ambiguous regions and low-coverage gaps may require different handling from simpler regions. Future work could therefore explore more adaptive filtering, region-aware read selection, or other strategies that respond to local evidence quality rather than using globally fixed thresholds.

11.5 Conclusion

Overall, the discussion supports a pipeline-level view of WhatsHap optimization. The experiments show that end-to-end phased recall depends strongly on upstream variant recovery, connectivity, and compounded realistic stress, while practical gains are best achieved through moderate caller-side and phasing-side filtering rather than aggressive tuning of a single parameter. These findings motivate the final conclusions summarized in Section 12.

12. Conclusion

This project investigated WhatsHap-based long-read phasing through a controlled synthetic benchmarking framework designed to balance realism, attribution, and experimental flexibility. The main objective was not only to measure phasing performance, but also to understand where performance is lost in practical end-to-end conditions and to identify optimization directions that are genuinely useful.

The study makes three main contributions.

First, it establishes a realistic and configurable benchmarking pipeline that supports both oracle and called regimes. This makes it possible to separate phaser-limited behaviour from full end-to-end behaviour and to attribute losses to variant recovery, phasing completeness, or phase correctness. The framework also supports multiple realism stressors, including duplicated regions, coverage dropout, correlated error bursts, variable read-length models, and truth indels with SNP-only safeguards. This experimental structure made it possible to move beyond simple performance reporting and instead study how different conditions affect WhatsHap-based phasing mechanisms.

Second, the experiments show that end-to-end performance is often limited less by the phasing algorithm itself than by the quality and overlap of informative heterozygous variants that survive upstream processing. In the oracle regime, WhatsHap remains highly accurate once correct sites are available, while the called regime often shows larger losses due to missing shared heterozygous sites and reduced connectivity. Among the isolated realism knobs, coverage dropout was the most impactful stressor, acting primarily through fragmentation and later also through reduced variant recovery. The duplication $\times$ dropout interaction further showed that realistic difficulty is often compounded rather than purely additive, with mixed caller-side and phaser-side weaknesses emerging most clearly under combined stress.

Third, the optimization study shows that useful but limited practical headroom exists. The strongest positive result came from caller-side base-quality tuning: reducing `call_min_baseq` to a moderate value improved call recall, shared heterozygous overlap, and end-to-end phased recall without materially harming precision. On the phasing side, the main lesson was to avoid overly strict filtering: `min_baseq = 20` was consistently harmful, whereas moderate values such as 10 lay on a stable performance plateau. Retained coverage also showed a practical runtime trade-off, with `max_coverage = 8` or `10` preserving performance while avoiding unnecessary compute. In contrast, caller and phasing mapping-quality thresholds provided little useful headroom and were effectively ruled out as major optimization levers.

The final recommended practical configuration from this study is therefore: - `call_min_baseq = 10` - `call_min_mapq = 20` (or default, since the effect was negligible) - `max_coverage = 8` - `min_mapq = 20` - `min_baseq = 10`

A slightly more runtime-biased alternative with `max_coverage = 6` performed almost identically on the main phasing metrics and may be acceptable when efficiency is prioritized.

The project also provides an algorithm-facing result beyond parameter tuning alone. The SNP-density scaling study showed that the solve stage becomes progressively more expensive as the phasing problem grows, but also that the balanced tuned configuration scales substantially better than the default configuration. This suggests that

practical tuning does not merely change final output metrics; it also changes the structure and difficulty of the residual wMEC-style problem presented to the solver.

At the same time, the project highlights an important methodological caution. Runtime profiling of the research pipeline showed that total phasing runtime is dominated by the custom readset-construction adaptor used for the experimental interface, rather than by the core solver itself. This adaptor was necessary to support oracle/called benchmarking, unified provenance, and controlled evaluation settings, but it is not the standard production-facing WhatsHap front-end. For this reason, the runtime results should be interpreted mainly as properties of the research pipeline and its controlled interface, not as a direct statement that the core WhatsHap solver is the dominant practical runtime bottleneck.

Overall, the central conclusion of this project is that WhatsHap is a strong and reliable phasing engine when informative variant evidence is available, but practical long-read phasing performance depends heavily on upstream variant recovery, connectivity, and the interaction of realistic stress conditions. Consequently, the most useful optimization directions are pipeline-level rather than purely solver-internal: preserve useful evidence, avoid overly conservative filtering, and improve the quality of the phasing problem that is ultimately passed to WhatsHap.

Future work can build on this study in several directions. One direction is to test the recommended configuration on broader and more realistic datasets, including real long-read data. Another is to investigate caller-side improvements more directly, since many end-to-end losses were shown to be overlap-limited rather than purely phaser-limited. A third is to examine the residual solver-facing problem more deeply, especially how connectivity, variant density, and evidence quality interact to affect the dynamic-programming stage. Finally, future work could compare the project-specific adaptor with the standard WhatsHap front-end directly, or optimize the adaptor separately if full pipeline runtime becomes a primary objective.

In summary, this project contributes a realistic benchmarking framework, a detailed attribution of WhatsHap-based phasing failures under controlled stress, and a practical set of experimentally supported optimization recommendations. These findings provide both a useful guide for practical WhatsHap use and a foundation for future work on more robust and efficient long-read phasing pipelines.